

UNIVERSIDADE REGIONAL DO NOROESTE DO
ESTADO DO RIO GRANDE DO SUL
CURSO DE CIÊNCIA DA COMPUTAÇÃO

JUAN PAULO FRICKE

ESTRUTURA EM VEZ DE ESCALA: AVALIAÇÃO DE *HARNESSES* EM NÍVEL DE REPOSITÓRIO PARA MAXIMIZAR A QUALIDADE DA SAÍDA DE LLMS RESTRITOS

Ijuí/RS
2026

JUAN PAULO FRICKE

ESTRUTURA EM VEZ DE ESCALA: AVALIAÇÃO DE *HARNESSES* EM NÍVEL DE REPOSITÓRIO PARA MAXIMIZAR A QUALIDADE DA SAÍDA DE LLMS RESTRITOS

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul - UNIJUÍ, como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Orientador(a): Adão Eurides de Souza Filho

Ijuí/RS

2026

JUAN PAULO FRICKE

ESTRUTURA EM VEZ DE ESCALA: AVALIAÇÃO DE *HARNESSES* EM NÍVEL DE REPOSITÓRIO PARA MAXIMIZAR A QUALIDADE DA SAÍDA DE LLMS RESTRITOS

Trabalho de Conclusão de Curso apresentado ao Curso de Ciência da Computação da Universidade Regional do Noroeste do Estado do Rio Grande do Sul - UNIJUÍ, como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação, submetido à aprovação da banca examinadora composta pelos seguintes membros:

Adão Eurides de Souza Filho
Orientador(a)

[MEMBRO DA BANCA]
Membro da Banca

Ijuí/RS, [DIA] de [MÊS] de 2026

AGRADECIMENTOS

[AGRADECIMENTOS: texto a ser escrito pelo autor]

LISTA DE FIGURAS

Figura 1 – Níveis cumulativos do *harness* e artefatos que cada um acrescenta à árvore . 23
Figura 2 – Desenho experimental: de um PR real à avaliação por testes ocultos e por qualidade de código 25
Figura 3 – Estágios do orquestrador, tratamento de falhas e papel do manifesto 27
Figura 4 – Resultado dos testes ocultos por tarefa, modelo e nível de *harness* (cinco modelos com a matriz completa) 29
Figura 5 – Variação da mediana de turnos e de *tokens* de entrada de cada nível em relação ao `none`, por modelo 31
Figura 6 – Efeito de cada nível contra `none` nas dimensões dos juízes (marcadores cheios: $p < 0{,}05$ no Wilcoxon exato) 33
Figura 7 – Nota geral dos juízes e casos de teste novos por execução, por modelo e nível de *harness* 34
Figura 8 – Linhas inseridas por execução, por categoria, modelo e nível de *harness* (rótulo: parcela de testes no código) 35
Figura 9 – Nota geral atribuída pelos dois juízes de cada dupla à mesma mudança (área do círculo proporcional ao número de mudanças) 36

LISTA DE QUADROS

Quadro 1 – As três camadas da análise de qualidade de código 17
Quadro 2 – Trecho das regras do enunciado de T07 (chamada da próxima senha), traduzido do original em inglês 26
Quadro 3 – Falhas sistemáticas observadas na rodada reduzida 30
Quadro 4 – Ameaças à validade, mitigação adotada e o que permanece em aberto . . . 41
Quadro 5 – Tarefas do *benchmark*, com categoria, porte, origem, tamanho da referência e observações 47
Quadro 6 – Dimensões da rubrica e âncoras das notas 5, 3 e 1 48
Quadro 7 – Incidentes da rodada reduzida (29 de outubro de 2026) e decisões tomadas 50

LISTA DE TABELAS

Tabela 1 – Modelos avaliados, *runners* e parâmetros 28
Tabela 2 – *Pass@1* por modelo e nível de *harness* (tarefas resolvidas em 14; IC 95% de Wilson entre colchetes) 28
Tabela 3 – Fração média de testes ocultos aprovados por modelo e nível 29
Tabela 4 – Efeito de cada nível contra `none` no *pass@1*, pareado por tarefa (+: só o nível passou; −: só o `none` passou) 30
Tabela 5 – Comportamento além do *pass@1*: execuções com regressão, aprovação no Pint e violações, por modelo 31
Tabela 6 – Efeito de cada nível contra `none` nas métricas de qualidade (Δ = média das diferenças pareadas por tarefa; Wilcoxon exato; 70 pares por nível, 63 a 70 nas métricas com dados ausentes) 33
Tabela 7 – Nota geral dos juízes (média de todos os juízes da execução, 1 a 5) e casos de teste novos por execução, por modelo e nível 34
Tabela 8 – Linhas inseridas por execução (média) em código de produção e em testes, por modelo e nível 35
Tabela 9 – Concordância entre juízes na nota geral e nas sete dimensões (todas as mudanças avaliadas pelos dois juízes da dupla) 37
Tabela 10 – Validade: correlação de Spearman entre medidas de camadas distintas (todas as execuções) 37
Tabela 11 – Regras de convenção, número de execuções em que cada uma se aplicou e em que foi cumprida (309 execuções) 49
Tabela 12 – Custo por execução (medianas) por modelo e nível 50

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface*
CLI	Interface de linha de comando (*Command-Line Interface*)
CSS	*Cascading Style Sheets*
HTTP	*Hypertext Transfer Protocol*
IC	Intervalo de confiança
JSON	*JavaScript Object Notation*
LLM	Grande modelo de linguagem (*Large Language Model*)
LSP	*Language Server Protocol*
PR	*Pull request*
TCC	Trabalho de Conclusão de Curso
UI	Interface de usuário (*User Interface*)
UNIJUÍ	Universidade Regional do Noroeste do Estado do Rio Grande do Sul

SUMÁRIO

APRESENTAÇÃO **9**
RESUMO **10**
ABSTRACT **11**
INTRODUÇÃO **12**
METODOLOGIA **14**
CLASSIFICAÇÃO DA PESQUISA 14
POR QUE UM *BENCHMARK* DE ABLAÇÃO 14
PROCEDIMENTOS 15
MÉTRICAS 16
ANÁLISE ESTATÍSTICA 17
MUDANÇA DE PLANEJAMENTO: DA RODADA COMPLETA À RODADA REDUZIDA 18
USO DE INTELIGÊNCIA ARTIFICIAL NO MÉTODO 18
REFERENCIAL TEÓRICO **19**
BENCHMARKS DE AGENTES DE CÓDIGO EM REPOSITÓRIOS REAIS 19
ENGENHARIA DE CONTEXTO E ARQUIVOS DE INSTRUÇÃO DE REPOSITÓRIO 20
FERRAMENTAS E AGENTES DE CÓDIGO 20
LLM COMO AVALIADOR 21
MÉTRICAS DE QUALIDADE DE CÓDIGO 22
DESENVOLVIMENTO **22**
O OBJETO DE ESTUDO: O *HARNESS* V1 22
O *BENCHMARK*: TAREFAS, ORÁCULO E AVALIAÇÃO 24
O ORQUESTRADOR 26
RESULTADOS: *PASS@1* E CUSTO 28
RESULTADOS: QUALIDADE DE CÓDIGO 31
DISCUSSÃO 38
AMEAÇAS À VALIDADE 39
CONSIDERAÇÕES FINAIS **40**

REFERÊNCIAS BIBLIOGRÁFICAS **43**
AS 14 TAREFAS DO *BENCHMARK* **47**
RUBRICA DOS JUÍZES **48**
REGRAS DE CONVENÇÃO C1 A C10 . . . . . **49**
DADOS COMPLEMENTARES DA RODADA REDUZIDA **50**

APRESENTAÇÃO

O presente trabalho foi elaborado no formato de artigo científico, conforme o Regimento do Trabalho de Conclusão de Curso do Curso de Ciência da Computação da UNIJUÍ, e segue o modelo de artigo de TCC adotado pelo curso. O artigo não foi submetido a periódico ou evento.

Os dados, os enunciados das tarefas, o *snapshot* do *harness* estudado e o código do orquestrador e da análise estão disponíveis publicamente no repositório do trabalho; o repositório-sujeito do experimento é privado.

ESTRUTURA EM VEZ DE ESCALA: AVALIAÇÃO DE *HARNESSES* EM NÍVEL DE REPOSITÓRIO PARA MAXIMIZAR A QUALIDADE DA SAÍDA DE LLMS RESTRITOS

SCAFFOLDING OVER SCALE: EVALUATING REPOSITORY-LEVEL HARNESSES TO MAXIMIZE OUTPUT QUALITY IN CONSTRAINED LLMS

Juan Paulo Fricke [1]

Adão Eurides de Souza Filho [2]

RESUMO

Agentes de código baseados em LLMs operam em repositórios que carregam a própria estrutura de apoio: regras sempre ativas (`AGENTS.md`, `CLAUDE.md`), *skills* de leitura sob demanda e configuração de ferramentas. Este trabalho pergunta se um *harness* desse tipo compensa a escala do modelo em tarefas reais. Foi construído um *benchmark* de ablação que reproduz 14 tarefas derivadas de *pull requests* de uma aplicação Laravel privada, com os testes do próprio PR como oráculo oculto, e um orquestrador reprodutível em *sandbox* Docker. Seis modelos (Claude Sonnet 5.5, Claude Haiku 5.5, Muse Spark 1.3, DeepSeek v4.1 flash, Beam e gpt-oss-20b local) executaram as 14 tarefas em quatro níveis cumulativos de *harness* (`none`, `agents`, `skills`, `full`), com uma repetição. A qualidade do código foi medida em três camadas: análise estática, dez regras de convenção do repositório e até três juízes LLM às cegas por mudança (309 execuções, 836 revisões). Quatro modelos resolveram 95% ou mais das tarefas já sem *harness*, e nenhum nível alterou o *pass@1* (McNemar exato, $p = 1{,}0$ em 70 pares por nível), enquanto o custo por execução subiu. O *harness*, porém, elevou a nota geral dos juízes (+0,31 a +0,34; $p < 0{,}001$), a de convenções (+0,46 a +0,52) e a de testes (+0,60 a +0,87), com dez casos de teste a mais por execução, sem alterar requisitos, segurança ou escopo. Nesta amostra, a escala explica a taxa de acerto e o *harness* explica parte da qualidade: ele muda como o código é escrito, não se a tarefa é resolvida.

Palavras-chave: agentes de código; grandes modelos de linguagem; engenharia de contexto; benchmark; qualidade de código; llm como avaliador.

[1] Estudante do Curso de Ciência da Computação – UNIJUÍ, Ijuí-RS, Brasil. E-mail: `juan.fricke@sou.unijui.edu.br`

[2] Professor Orientador, Mestre – UNIJUÍ, Ijuí-RS, Brasil. E-mail: `adao.eurides@unijui.edu.br`

ABSTRACT

LLM-based coding agents increasingly operate in repositories that ship their own scaffolding: always-on rules (`AGENTS.md`, `CLAUDE.md`), on-demand skills and tool configuration. This work asks whether such a repository-level harness compensates for model scale on real tasks. An ablation benchmark was built that replays 14 tasks derived from pull requests of a private Laravel application, using each PR's own tests as a hidden oracle, together with a reproducible orchestrator that runs every cell in a Docker sandbox. Six models (Claude Sonnet 5.5, Claude Haiku 5.5, Muse Spark 1.3, DeepSeek v4.1 flash, Beam and a local gpt-oss-20b) ran the 14 tasks at four cumulative harness levels (`none`, `agents`, `skills`, `full`), with one repetition. Code quality was measured in three layers: static analysis, ten convention rules taken from the repository and up to three blind LLM judges per change (309 runs, 836 reviews). Four models solved 95% or more of the tasks without any harness, and no level changed pass@1 detectably (exact McNemar, $p = 1.0$ over 70 pairs per level), while the cost per run rose. The harness did, however, raise the judges' overall score (+0.31 to +0.34; $p < 0.001$), the conventions score (+0.46 to +0.52) and the tests score (+0.60 to +0.87), with about ten more test cases per run, leaving requirements, security and scope unchanged. In this sample, scale explains the pass rate and the harness explains part of the quality: it changes how the code is written, not whether the task gets solved.

Keywords: coding agents; large language models; context engineering; benchmark; code quality; llm-as-a-judge.

INTRODUÇÃO

Agentes de código baseados em grandes modelos de linguagem (LLMs, do inglês *Large Language Models*) deixaram de sugerir linhas isoladas durante a escrita do código para serem diretamente o principal motor de escrita: leem arquivos, editam código, executam testes e iteram até considerar uma tarefa concluída. Ferramentas como o Claude Code (Anthropic, 2026a) e o OpenCode (OpenCode, 2026) são duas das mais utilizadas ferramentas desse modo de trabalho em linha de comando, e *benchmarks* como o SWE-bench (Jimenez et al., 2024) mostram que a taxa de resolução de tarefas reais cresce com a capacidade do modelo, ou seja, modelos maiores e mais bem treinados conseguem concluir tarefas de codificação de forma autônoma com maior eficiência. Essa capacidade, porém, tem preço: os modelos de ponta custam mais por *token*, dependem de cotas de uso de planos de assinatura mensal e nem sempre podem ser usados, seja por restrições de custo, de privacidade dos dados ou pela necessidade de execução local em *hardware* modesto.

Em paralelo, consolidou-se a prática de os próprios repositórios carregarem uma estrutura de apoio destinada à leitura exclusiva dos agentes de terminal que neles trabalham. Arquivos de instrução como `AGENTS.md` (AGENTS.md, 2025) e `CLAUDE.md` (Anthropic, 2025a) descrevem convenções, comandos e regras do projeto; *skills*, sugestões e guias para o código ficam disponíveis para leitura; e arquivos de configuração habilitam o uso de servidores de linguagem (LSP, *Language Server Protocol*) e outras ferramentas. Neste trabalho, esse conjunto de artefatos versionados no repositório é chamado de *harness*[3]. A hipótese implícita nessa prática, resumida no lema *scaffolding over scale*, é que uma boa estrutura de apoio compensa, ao menos em parte, a escala do modelo: um modelo menor bem orientado produziria resultados comparáveis aos de um modelo maior sem orientação. Apesar da adoção ampla, a evidência controlada sobre esse efeito ainda é escassa. O que circula, em geral, são relatos de praticantes, como o de Akita (2026), que compara um mesmo modelo em *harnesses* diferentes em seu *benchmark* próprio e conclui que, para um modelo de fronteira, o *harness* é ruído, ao passo que, para um modelo fraco, ele pode mudar o resultado. Em resumo, estudos empíricos da eficácia do *harness* e do tipo de *harness* são escassos, e muitas das convenções são criadas por sentimento ou por impressões de desenvolvedores e de repositórios influentes.

O problema que orienta esta pesquisa é, portanto, o seguinte: um *harness* em nível de repositório pode compensar a escala do modelo? A pergunta desdobra-se em três: (i) o *harness* aumenta a taxa com que agentes resolvem tarefas reais, medida por testes que o agente não vê? (ii) o *harness* muda a qualidade do código produzido, mesmo quando a tarefa já é resolvida? e (iii) qual camada do *harness*, se alguma, carrega o efeito observado?

O objetivo geral deste trabalho é avaliar, por meio de um experimento de ablação, o

[3] O termo *harness*, sem tradução consolidada em português, designa aqui o conjunto de regras, *skills* e configuração de ferramentas que o repositório oferece ao agente. A "estrutura" do título traduz *scaffolding* e refere-se ao mesmo conjunto, visto como apoio externo ao modelo.

efeito de um *harness* de repositório real sobre a saída de agentes de código de diferentes escalas, em tarefas derivadas do histórico desse mesmo repositório. Os objetivos específicos são:

1. construir um *benchmark* que reproduza *pull requests* reais de uma aplicação Laravel, usando os testes de cada PR como oráculo oculto;
2. definir níveis cumulativos de *harness* a partir de um *snapshot* congelado do repositório e aplicá-los de forma idêntica a todas as tarefas;
3. executar seis modelos de escalas distintas em todas as tarefas e níveis, em ambiente isolado e reprodutível, registrando acerto, custo e comportamento;
4. medir a qualidade do código produzido em três camadas independentes: análise estática, aderência às convenções do repositório e revisão por juízes LLM às cegas;
5. analisar estatisticamente o efeito de cada nível, pareado por tarefa dentro de cada modelo, e discutir os resultados à luz da hipótese *scaffolding over scale*.

As contribuições do trabalho são três. A primeira é um *benchmark* de ablação sobre PRs reais com oráculo oculto, cujo repositório-sujeito é privado e, portanto, pouco sujeito à contaminação dos dados de treinamento dos modelos, que usam dados e repositórios públicos para treinamento. A segunda é um orquestrador reprodutível que executa a matriz modelo $\times$ tarefa $\times$ nível em *sandbox* Docker, com isolamento da árvore de trabalho, detecção de contaminação, tratamento de limites de taxa e retomada sem duplicar células. A terceira é uma análise de qualidade de código em três camadas, com até três juízes LLM por mudança, de três famílias, um controle de autopreferência, validação cruzada entre camadas e medidas de concordância entre juízes. Os dados resultantes compreendem 309 execuções avaliadas, de um desenho de 336 células, e 836 revisões de juízes.

Antecipando os resultados, o experimento mostrou que quatro dos seis modelos resolveram 95% ou mais das tarefas já sem nenhum *harness*, que nenhum nível alterou a taxa de acerto de forma detectável e que o *harness* encareceu cada execução. Em contrapartida, o *harness* elevou de forma consistente as notas de convenções e de testes atribuídas pelos juízes e o número de casos de teste escritos, sem alterar requisitos, segurança ou escopo. O efeito concentrou-se nos modelos de desempenho intermediário e não apareceu nos mais fracos.

O restante do artigo está organizado da seguinte forma. A seção 2 classifica a pesquisa e descreve os procedimentos, as métricas, os testes estatísticos e a mudança de planejamento ocorrida durante o experimento. A seção 3 situa o trabalho em cinco eixos: *benchmarks* de agentes de código, engenharia de contexto, ferramentas de agentes, LLM como avaliador e métricas de qualidade de código. A seção 4 apresenta o objeto de estudo, o *benchmark* e o orquestrador, e em seguida os resultados de *pass@1*, custo e qualidade, a discussão e as ameaças à validade. A seção 5 retoma os objetivos, resume as contribuições e as limitações e aponta trabalhos futuros.

METODOLOGIA

Esta seção descreve o delineamento da pesquisa: sua classificação, os procedimentos seguidos, as métricas e os testes estatísticos adotados, a mudança de planejamento ocorrida durante o experimento e o papel das ferramentas de inteligência artificial no método. A execução técnica de cada etapa é detalhada na seção 4.

CLASSIFICAÇÃO DA PESQUISA

Quanto à natureza, trata-se de pesquisa aplicada, pois busca responder a uma pergunta prática da engenharia de software com LLMs: se e como vale a pena investir em um *harness* de repositório e qual é o seu efeito sobre diferentes níveis de modelos de IA. Quanto à abordagem, é quantitativa: todas as variáveis de resposta são medidas numericamente e analisadas com estatística descritiva e inferencial. Quanto aos objetivos, é explicativa, no sentido de Gil (2017), pois procura identificar o efeito de um fator manipulado sobre variáveis de resposta. Quanto aos procedimentos, é um experimento controlado, na terminologia de Wohlin et al. (2012), organizado como um estudo de ablação: a partir de um *harness* completo, camadas são removidas progressivamente e o efeito de cada remoção é medido.

O fator manipulado é o nível do *harness*, com quatro valores cumulativos (`none`, `agents`, `skills` e `full`, descritos na subseção 4.1). O modelo de linguagem é um fator de bloco: as comparações entre níveis são feitas dentro de cada modelo, nunca entre modelos, porque os modelos foram executados por *runners* diferentes. A tarefa é a unidade de pareamento: cada tarefa é executada por cada modelo em cada nível, de modo que o efeito de um nível é estimado pela diferença, tarefa a tarefa, em relação ao nível `none`. Uma célula do experimento é, portanto, a tripla modelo $\times$ tarefa $\times$ nível, e uma execução (*run*) é a materialização de uma célula.

POR QUE UM *BENCHMARK* DE ABLAÇÃO

Em aprendizado de máquina e engenharia de sistemas, ablação é a técnica de remover (ou, de forma equivalente, acrescentar) um componente de um sistema, mantendo todo o resto fixo, para medir quanto aquele componente contribui para o resultado. Um *benchmark* de ablação, portanto, não pergunta “qual modelo é melhor”, e sim “o que muda quando este componente está presente?”. Aqui, o componente é o *harness* do repositório, e os níveis `none`, `agents`, `skills` e `full` acrescentam camadas cumulativas sobre as mesmas tarefas, o mesmo modelo e o mesmo ambiente.

A escolha decorre da pergunta de pesquisa, que é causal. Um *benchmark* de ordenação, como o SWE-bench, varia o modelo e mantém a tarefa, o que serve para classificar modelos. Mas comparar posições de uma tabela de classificação não isola o *harness*, porque os participantes

diferem ao mesmo tempo em modelo, ferramenta, *prompt* e configuração, de modo que qualquer diferença fica atribuída a tudo junto. Um estudo observacional sobre repositórios públicos com e sem `AGENTS.md` teria o mesmo problema, pois repositórios que adotam o arquivo diferem em maturidade, tamanho e cobertura de testes. Um estudo com desenvolvedores controlaria melhor o contexto, mas custaria muito mais e traria a variância do humano para dentro da medida.

A ablação resolve esses problemas com o pareamento: cada tarefa é executada pelo mesmo modelo em todos os níveis, de modo que a dificuldade da tarefa e as habilidades do modelo se cancelam na diferença. Isso importa para uma amostra pequena, de 14 tarefas, em que a variação entre tarefas seria maior que o efeito procurado se as comparações fossem entre grupos independentes. O custo do desenho é que os níveis são cumulativos e não fatoriais, o que impede separar interações entre camadas e limita a generalização a um único repositório; esse limite é retomado nas considerações finais.

PROCEDIMENTOS

O experimento seguiu oito etapas, executadas entre 1º e 29 de outubro de 2026.

Seleção das tarefas a partir de PRs. O repositório-sujeito é uma aplicação web de gestão de filas de atendimento escrita em Laravel (PHP) com Bulma CSS e Livewire, privada e desenvolvida pelo autor. Cada tarefa reproduz um *pull request* (PR) ou *branch* desse repositório: a árvore no *commit* base do PR é o ponto de partida do agente, um enunciado escrito a partir do PR e da documentação de domínio é a tarefa, e os arquivos de teste adicionados ou modificados pelo PR são o oráculo oculto, copiados para a árvore apenas no momento da avaliação. O enunciado carrega o contrato público de que os testes dependem (nomes de classes, métodos, rotas, chaves de configuração), para que a tarefa meça engenharia e não adivinhação de nomes; os pontos em que o acoplamento permanece forte são registrados por tarefa. Dois PRs grandes, sem *commit* intermediário aproveitável, foram mantidos como tarefas compostas de referência e divididos em subtarefas que compartilham base e solução, mas avaliam subconjuntos separáveis dos testes. O conjunto final tem 14 tarefas: nove PRs ou *branches* inteiros e cinco subtarefas das duas compostas (subseção 4.2 e Apêndice A).

Definição dos níveis. Os quatro níveis são cumulativos: `none` não contém nenhum artefato do *harness*; `agents` adiciona as regras sempre ativas (`AGENTS.md` e `CLAUDE.md`); `skills` adiciona o diretório `ai/` com *skills*, comandos, regras e modelos de código para leitura sob demanda; `full` adiciona a configuração dos *runners* e dos servidores de linguagem (`opencode.json` e `.claude/settings.json`). O *harness* é aplicado em um *commit* separado, marcado com a *tag* `bench/start`, de modo que o *diff* do agente nunca inclui os arquivos do *harness*.

Modelos e *runners*. Foram avaliados seis modelos de escalas e origens distintas. Claude Sonnet 5.5 e Claude Haiku 5.5 foram executados exclusivamente pelo Claude Code 2.1.312, com

esforço de raciocínio `high`. Muse Spark 1.3, DeepSeek v4.1 flash e Beam (plano gratuito) foram executados pelo OpenCode v2.2.0 por meio do provedor OpenCode Go, com os parâmetros padrão do provedor. O gpt-oss-20b (OpenAI, 2025) foi executado localmente, pelo OpenCode contra um servidor llama.cpp (Gerganov et al., 2026) em uma GPU Radeon RX 9070 de 16 GB, com quantização MXFP4, contexto de 128 mil *tokens* e a amostragem recomendada pelo fabricante. Os modelos da Anthropic rodaram apenas no Claude Code, nunca por intermédio do OpenCode, para que o *runner* nativo de cada família fosse usado. Em cada execução, as mensagens da sessão foram conferidas para garantir que somente o modelo sob teste respondeu.

Execução orquestrada e avaliação automática. Um orquestrador escrito em Python para este trabalho prepara cada célula, executa o agente em um contêiner Docker com acesso apenas à própria árvore e avalia o resultado em um segundo contêiner, copiando os testes ocultos e executando o Pint, a suíte completa e os testes ocultos isoladamente. Todos os agentes recebem o mesmo *prompt*: um preâmbulo não interativo fixo seguido do enunciado sem alterações, com 30 minutos de limite; a ordem das células foi embaralhada com semente fixa (20261028) e cada execução registra versões, imagens e o *hash* do *prompt*. A subseção 4.3 detalha o orquestrador.

Análise de qualidade em três camadas. Como a taxa de acerto ficou próxima do teto para a maioria dos modelos, uma segunda fase perguntou se o *harness* muda *como* o código é escrito. Essa fase trabalha sobre os *patches* e as árvores deixadas pelas execuções, sem rodar agentes de novo, em três camadas: (1) métricas estáticas, com Larastan nível 5 (Larastan, 2026) para contar erros novos em relação à base, complexidade ciclomática de McCabe por função, casos de teste escritos e distância da solução de referência; (2) dez regras heurísticas (C1 a C10) derivadas do próprio `AGENTS.md`, cada uma aplicada apenas quando a mudança toca o seu assunto; (3) revisão às cegas por até três juízes LLM por mudança, Claude Opus 5.5, Muse Spark 1.3 e DeepSeek v4.1 flash, seguindo uma rubrica de sete dimensões com notas de 1 a 5 (Apêndice B). Cada mudança recebeu dois juízes de famílias distintas da do modelo que a escreveu e, quando existe, o juiz da própria família; a nota da execução é a média de todos os seus juízes, e a nota do juiz da família é comparada à dos dois de fora como controle de autopreferência. O Quadro 1 resume as três camadas. Um piloto com duas tarefas (46 execuções, 92 revisões) validou o método antes da rodada completa (309 execuções, 836 revisões), e todas as violações da camada 2 apontadas no piloto foram lidas manualmente contra o código.

Análise estatística. Descrita na subseção 2.5.

MÉTRICAS

os testes ocultos passam em uma única tentativa, na definição de Chen et al. (2021) para $k = 1$. Como métrica contínua, registra-se a fração de testes ocultos aprovados. Regressões são testes fora do conjunto oculto que falham além dos que já falhavam na árvore intocada, medidos por tarefa com um agente nulo como linha de base. O custo é medido por turnos do agente,

Quadro 1 – As três camadas da análise de qualidade de código

Camada	O que mede	Ferramenta	Saída por execução
1. Estática	Erros novos de análise estática em relação à base; complexidade ciclomática por função; casos de teste e linhas escritas, separadas em produção e testes; distância da solução de referência	Larastan nível 5, analisador de complexidade em PHP, contagem de testes Pest, índice de Jaccard dos arquivos de produção	`larastan_new_errors`, `cc_mean`, `test_cases_added`, `ref_files_jaccard`
2. Convenções	Dez regras do `AGENTS.md` (C1 a C10), cada uma aplicada só quando a mudança toca o seu assunto	Heurísticas sobre o *patch*, com evidência guardada por violação	Escore de 0 a 1 (fração das regras aplicáveis cumpridas)
3. Juízes	Sete dimensões (requisitos, convenções, segurança, testes, simplicidade, legibilidade, escopo) e nota geral, de 1 a 5, com evidência obrigatória	Até três juízes LLM às cegas (Opus 5.5, Muse Spark 1.3, DeepSeek v4.1 flash): dois de famílias distintas da do autor e, quando existe, o da própria família; rubrica e esquema JSON	`review.json` por juiz; média de todos os juízes por dimensão

Fonte: elaborado pelo autor (2026).

tokens de entrada (incluindo leitura de *cache*) e de saída e tempo de parede; turnos e *tokens* são comparáveis entre níveis de um mesmo modelo, enquanto o tempo depende do provedor e da concorrência. A qualidade é medida pelas três camadas descritas acima: escore de convenções (fração das regras aplicáveis cumpridas), erros novos do Larastan, complexidade média, casos de teste novos, linhas inseridas em código de produção e em testes, índice de Jaccard entre os arquivos de produção tocados e os da referência, e as notas dos juízes em cada dimensão e na nota geral, com a média de todos os juízes da execução.

ANÁLISE ESTATÍSTICA

Todas as comparações são pareadas por tarefa, dentro de cada modelo, contra o nível `none`. Para o *pass@1*, um resultado binário, usa-se o teste exato de McNemar (McNemar, 1947), que considera apenas as tarefas discordantes entre os dois níveis; os intervalos de confiança de 95% das proporções são de Wilson (Wilson, 1927). Para as métricas contínuas e ordinais de qualidade, usa-se o teste exato dos postos sinalizados de Wilcoxon (Wilcoxon, 1945), adequado a amostras pequenas sem suposição de normalidade. A concordância entre juízes é medida pelo κ ponderado de Cohen (1968), com pesos quadráticos, interpretado pela escala de Landis e Koch (1977), e pela correlação de postos de Spearman (Spearman, 1904). A autopreferência é medida, em cada mudança, pela diferença entre a nota do juiz da família do autor e a média dos dois juízes de fora (Wilcoxon exato pareado), descontada a leniência geral desse juiz nas mudanças de outras famílias. A validade das notas dos juízes é verificada pela correlação de Spearman entre a nota e medidas independentes (acerto nos testes ocultos, escore de convenções e erros do Larastan). O agregado entre modelos usa os 70 pares por nível dos cinco modelos com a matriz completa (5 modelos $\times$ 14 tarefas), excluindo o Beam, que ficou incompleto. Não foi aplicada

correção para comparações múltiplas; o número de comparações é informado e a leitura dos *p*-valores considera esse fato (subseção 4.7).

MUDANÇA DE PLANEJAMENTO: DA RODADA COMPLETA À RODADA REDUZIDA

O plano original previa cinco modelos, 14 tarefas, quatro níveis e três repetições (840 células), com os modelos GLM 5.3 flash e MiMo v2.6 flash no lugar de Muse Spark, DeepSeek e Beam. A rodada foi iniciada em 28 de outubro de 2026 às 18h51 e cancelada às 19h40, depois de 41 células, quando a estimativa de duração com tempos reais ultrapassou 30 horas; o gargalo eram os dois modelos mais lentos do provedor, com cerca de 75 e 45 *tokens* por segundo em sessões agênticas. Com o limite de 12 horas disponível, optou-se por uma rodada reduzida: uma repetição em vez de três, os três modelos mais rápidos do provedor (Muse Spark, Beam e DeepSeek, medidos em 376, 339 e 280 *tokens* por segundo isoladamente) e os dois modelos locais. Essa decisão foi registrada com data e justificativa no diário do orquestrador. A consequência metodológica é que a repetição vem das tarefas, não de execuções repetidas da mesma célula: a variação entre tarefas tende a superar a variação entre execuções da mesma tarefa, mas a variância de um mesmo modelo na mesma célula não é estimada. Durante a rodada reduzida, o segundo modelo local (Qwen 3.6 35B-A3B) foi retirado por lentidão (cerca de 23 minutos por célula, o que levaria 20 horas) e o provedor do Beam ficou fora do ar; o Beam foi encerrado com 29 das 56 células e é apresentado apenas de forma descritiva. O desenho final tem 6 modelos $\times$ 14 tarefas $\times$ 4 níveis $\times$ 1 repetição (336 células).

USO DE INTELIGÊNCIA ARTIFICIAL NO MÉTODO

A inteligência artificial participa deste trabalho de três maneiras, que convém distinguir. Primeiro, os modelos de linguagem são o *objeto* do experimento: os seis modelos avaliados escrevem o código que é medido. Segundo, modelos são parte do *instrumento* de medida: Claude Opus 5.5, Muse Spark 1.3 e DeepSeek v4.1 flash atuam como juízes na terceira camada da análise de qualidade, sob as salvaguardas descritas na subseção 4.5. Terceiro, modelos foram *ferramentas* de construção: o orquestrador, os *scripts* de análise e os relatórios intermediários foram desenvolvidos com o auxílio do Claude Code (Claude Opus 5.5), sob direção e revisão do autor; o código do repositório-sujeito foi escrito, em parte, com o Muse Spark, o que é discutido como ameaça à validade; e o texto deste artigo foi redigido com o auxílio do Claude Fable 5.1 e revisado pelo autor, que responde integralmente pelo conteúdo. A declaração formal de uso consta ao fim do trabalho.

REFERENCIAL TEÓRICO

Esta seção organiza o estado da arte em cinco eixos que sustentam o desenho do experimento: a avaliação de agentes de código em repositórios reais, a engenharia de contexto e os arquivos de instrução de repositório, as ferramentas que executam esses agentes, o uso de LLMs como avaliadores e as métricas de qualidade de código. Ao fim de cada eixo, indica-se o que este trabalho acrescenta.

BENCHMARKS DE AGENTES DE CÓDIGO EM REPOSITÓRIOS REAIS

A avaliação de geração de código por LLMs começou com problemas isolados. Chen et al. (2021) introduziram o HumanEval e a métrica *pass@k*, a probabilidade de que ao menos uma entre k amostras passe nos testes do problema, e Austin et al. (2021) seguiram o mesmo formato com o MBPP. Liu et al. (2023) mostraram, porém, que os testes desses conjuntos eram insuficientes: ao ampliá-los no EvalPlus, a taxa de acerto de diversos modelos caiu de forma expressiva, o que evidencia a dependência de qualquer *benchmark* da qualidade do seu oráculo.

O passo para tarefas em escala de repositório foi dado pelo SWE-bench (Jimenez et al., 2024), que reúne *issues* reais de projetos Python no GitHub e avalia a correção pelos testes do PR que as resolveu, separando testes que devem passar a falhar-para-passar dos que devem continuar passando. A versão SWE-bench Verified (OpenAI, 2024) filtrou manualmente os itens com enunciados subespecificados ou testes injustos, reconhecendo que parte das falhas medidas vinha do *benchmark* e não dos modelos. Sobre essa base, Yang et al. (2024) mostraram, com o SWE-agent, que a interface entre o agente e o computador (comandos de busca, edição com verificação sintática, visualização paginada de arquivos) altera substancialmente a taxa de resolução para um mesmo modelo, e Wang et al. (2024b) consolidaram no OpenHands uma plataforma aberta para agentes generalistas de desenvolvimento. Em paralelo, trabalhos de completação em nível de repositório, como o de Shrivastava, Larochelle e Tarlow (2023) e o RepoCoder de Zhang et al. (2023), demonstraram que selecionar contexto relevante do restante do repositório melhora a geração, o que fundamenta a ideia de que o repositório pode e deve informar o modelo.

Esses trabalhos têm em comum o uso de repositórios públicos, muitas vezes presentes nos dados de treinamento dos modelos avaliados, e a concentração em Python. Este trabalho acrescenta um *benchmark* construído sobre um repositório privado em PHP/Laravel, cujos *commits* não são acessíveis aos modelos, com o mesmo princípio de oráculo oculto do SWE-bench, e usa esse *benchmark* não para ordenar modelos, mas para medir, por ablação, o efeito de uma intervenção no repositório: o *harness*.

ENGENHARIA DE CONTEXTO E ARQUIVOS DE INSTRUÇÃO DE REPOSITÓRIO

Agentes de código seguem, em essência, o laço de raciocínio e ação proposto por Yao et al. (2023): o modelo alterna entre pensar, chamar uma ferramenta e observar o resultado, acumulando tudo na janela de contexto. Como essa janela é finita e o desempenho degrada quando a informação relevante fica dispersa ou distante (Liu et al., 2024), a seleção do que entra no contexto passou a ser uma disciplina própria, chamada de engenharia de contexto (Anthropic, 2025b). A prática recomendada é separar o que deve estar sempre presente (regras curtas, comandos de construção e teste, convenções) do que deve ser lido sob demanda, quando a tarefa o exigir.

Essa separação materializou-se em arquivos versionados no repositório. O `CLAUDE.md` do Claude Code é lido automaticamente no início de cada sessão e deve conter o mínimo necessário para o agente operar no projeto (Anthropic, 2025a); o `AGENTS.md` propõe-se como formato aberto e neutro em relação à ferramenta para o mesmo fim (AGENTS.md, 2025). *Skills* são documentos mais longos, organizados por assunto, que o agente abre quando a tarefa toca aquele assunto; a sua existência só é anunciada nas regras sempre ativas. A configuração de ferramentas, por fim, habilita servidores de linguagem e outros recursos que mudam o que o agente consegue observar sem ler arquivos inteiros.

A literatura sobre esses artefatos é, em grande parte, prescritiva: descreve o que colocar neles, mas raramente mede o efeito em condições controladas. O relato de Akita (2026) aponta que, em modelos de fronteira, trocar o *harness* quase não altera a nota no seu *benchmark*, mas o texto é um artigo de opinião, com poucos detalhes sobre repetições e margem de erro, e compara programas-cliente de agente, não arquivos de instrução de repositório. A documentação da Anthropic (2026b) segue a mesma linha de orientação sem dados abertos: afirma que *prompts* escritos para o modelo anterior devem funcionar sem mudanças no Claude Opus 5.5, e, ao mesmo tempo, recomenda reavaliar nos próprios testes o nível de esforço e o apoio construído para entradas visuais em vez de reaproveitar a configuração antiga. As melhorias são descritas como resultados de testes internos da Anthropic, sem publicar as medidas nem o método, o que impede a reprodução. Este trabalho acrescenta uma ablação em quatro níveis cumulativos, aplicada a um *harness* real e congelado, com o mesmo enunciado e o mesmo ambiente em todos os níveis, de modo a atribuir qualquer diferença observada à camada adicionada.

FERRAMENTAS E AGENTES DE CÓDIGO

O Claude Code (Anthropic, 2026a) e o OpenCode (OpenCode, 2026) são agentes de terminal: recebem uma tarefa em linguagem natural, dispõem de ferramentas de leitura, busca, edição e execução de comandos, e trabalham até concluir. Ambos leem arquivos de instrução do repositório e podem ser executados em modo não interativo, o que os torna adequados a um experimento automatizado. Eles diferem no *prompt* de sistema, no conjunto de ferramentas e na

gestão de contexto, o que torna a comparação entre modelos executados por *runners* distintos uma comparação de pares modelo-*runner*, e não de modelos isolados.

A escolha do modelo, por sua vez, envolve uma relação entre capacidade e custo descrita pelas leis de escala: o desempenho melhora de forma previsível com parâmetros, dados e computação (Kaplan et al., 2020), e há um tamanho ótimo de modelo para cada orçamento de treinamento (Hoffmann et al., 2022). Para quem usa os modelos, isso se traduz em famílias com variantes de tamanhos e preços distintos e em modelos abertos, como o gpt-oss-20b (OpenAI, 2025), que podem ser executados localmente com *runtimes* como o llama.cpp (Gerganov et al., 2026), com quantização e as limitações de *hardware* que isso implica.

Este trabalho acrescenta uma comparação do mesmo *harness* em seis modelos, de uma família comercial de ponta a um modelo aberto quantizado em GPU de consumo, executados por dois *runners* sob um mesmo protocolo, com a comparação de níveis sempre feita dentro de cada par modelo-*runner*.

LLM COMO AVALIADOR

Quando a métrica de interesse não é binária, como a qualidade de um *pull request*, a avaliação humana é cara e a avaliação automática tradicional é limitada. Zheng et al. (2023) propuseram usar LLMs fortes como juízes e mostraram, com o MT-Bench e a Chatbot Arena, que o GPT-4 concorda com avaliadores humanos em nível comparável ao da concordância entre humanos. O mesmo trabalho catalogou vieses do método: viés de posição (a preferência pela resposta apresentada primeiro), viés de verbosidade (a preferência por respostas mais longas) e autopreferência (a preferência por respostas do próprio modelo). Wang et al. (2024a) mostraram que a simples troca de ordem das respostas pode inverter o veredito, e Panickssery, Bowman e Feng (2024) demonstraram que a autopreferência está ligada à capacidade do modelo de reconhecer os próprios textos. Para o viés de verbosidade, Dubois et al. (2024) propuseram controlar o comprimento na análise. No domínio de código, Zhuo (2024) mostraram que LLMs instruídos com critérios explícitos avaliam utilidade e correção funcional com correlação superior à de métricas baseadas em similaridade textual.

Este trabalho aplica essas lições ao desenho dos juízes: usa até três juízes por mudança, de três famílias, e mede a autopreferência comparando o juiz da mesma família do modelo que escreveu o código com os dois de fora, em vez de apenas excluí-lo; avalia cada mudança isoladamente, com nota absoluta e evidência obrigatória por dimensão, para evitar o viés de posição; instrui explicitamente a não recompensar o comprimento; e, sobretudo, mede a concordância entre juízes e a correlação das notas com medidas independentes (testes ocultos, regras de convenção e análise estática) em vez de assumir a sua validade. A ligação entre um juiz (Muse Spark) e o repositório-sujeito é discutida como ameaça à validade.

MÉTRICAS DE QUALIDADE DE CÓDIGO

Qualidade de código é um construto multidimensional. A norma ISO/IEC 25010 (ISO/IEC, 2011) organiza a qualidade de produto em características como adequação funcional, segurança e manutenibilidade, e esta última desdobra-se em analisabilidade, modificabilidade e testabilidade. Medidas estruturais tentam aproximar essas características: a complexidade ciclomática de McCabe (1976) conta os caminhos linearmente independentes de uma função e é usada há décadas como indicador de dificuldade de teste e de manutenção. A análise estática complementa essas medidas ao apontar erros de tipo, chamadas a métodos inexistentes e outros defeitos sem executar o código; no ecossistema PHP, o PHPStan e sua extensão para Laravel, o Larastan (Larastan, 2026), cumprem esse papel em níveis de rigor crescentes.

Para código gerado por LLMs, a literatura tem apontado que correção funcional não implica qualidade. Pearce et al. (2022) encontraram vulnerabilidades em cerca de 40% dos cenários em que o GitHub Copilot produziu código relevante para segurança, o que motiva uma dimensão de segurança separada da de requisitos. Práticas de engenharia como o desenvolvimento orientado a testes (Beck, 2002) e os princípios de legibilidade e simplicidade de Martin (2008) fornecem o vocabulário das convenções que os repositórios codificam em seus arquivos de instrução e que revisores humanos cobram em *pull requests*.

Este trabalho acrescenta uma operacionalização em três camadas que combina medidas objetivas (Larastan, complexidade, testes escritos), regras heurísticas derivadas das convenções do próprio repositório e julgamento por LLM em sete dimensões alinhadas à ISO/IEC 25010 e às convenções do projeto, com verificação cruzada entre as camadas. A escolha de medir o que o *harness* prescreve (testes, convenções, autorização, simplicidade) é deliberada: é nessas dimensões que uma intervenção textual tem como agir.

DESENVOLVIMENTO

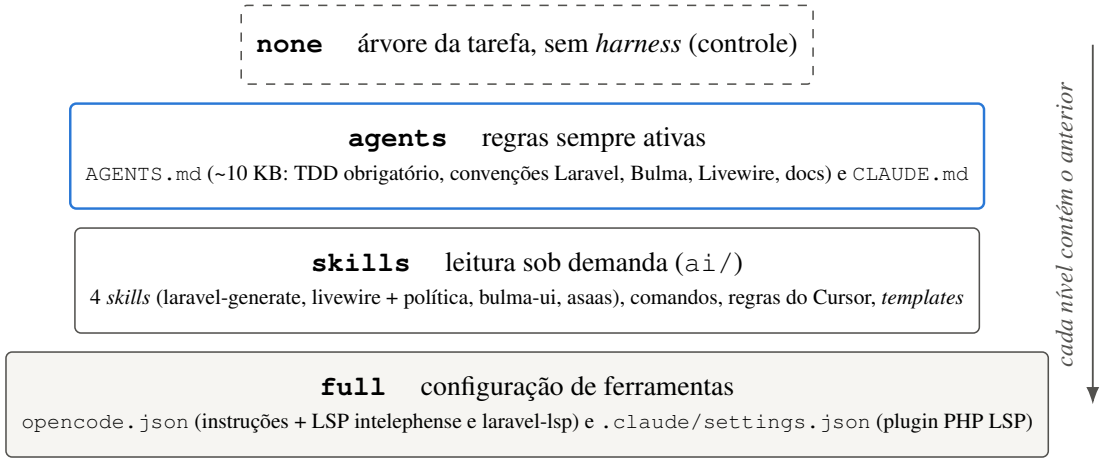
Esta seção apresenta o objeto de estudo, o *benchmark* e o orquestrador construídos para avaliá-lo e, em seguida, os resultados de *pass@1* e custo, os resultados de qualidade de código, a discussão e as ameaças à validade.

O OBJETO DE ESTUDO: O *HARNESS* V1

O *harness* estudado é o conjunto de artefatos que o repositório-sujeito oferece aos agentes de código, congelado na *tag* `tcc-baseline-v1`. Ele tem três camadas, que correspondem aos três níveis de tratamento do experimento (Figura 1).

A primeira camada são as regras sempre ativas. O arquivo `AGENTS.md`, com cerca de 10 KB, declara-se autossuficiente e reúne tudo o que um agente deve seguir no projeto: a

Figura 1 – Níveis cumulativos do *harness* e artefatos que cada um acrescenta à árvore



Fonte: elaborado pelo autor (2026).

disciplina de "escrever menos, corretamente" (reutilizar antes de criar, preferir recursos nativos, nunca cortar validação ou segurança); o desenvolvimento orientado a testes como obrigação ("nenhum código de produção sem teste"), com execução dos testes por um comando fixo; convenções Laravel (geradores do `artisan`, migrações geradas e reversíveis, validação com `#[Validate]` em componentes Livewire, autorização por *policies*, escritas de domínio em classes de ação sob `app/Actions`, nenhuma camada de serviço ou repositório, `$fillable` ou `$guarded` definidos de propósito); convenções de interface com Bulma (CSS puro, sem bibliotecas de componentes em JavaScript, componentes Blade com escape automático); uma política de segurança para Livewire que trata toda propriedade pública como campo controlável pelo cliente e exige autorização em toda ação; regras para integrações de pagamento e para pontos de injeção de *prompt*; e a obrigação de manter a documentação e o `CHANGELOG.md` em sincronia. A Listagem 1 reproduz os trechos que mais importam para os resultados. O arquivo `CLAUDE.md` contém uma única linha que importa o `AGENTS.md`, para que o Claude Code leia as mesmas regras.

A segunda camada é o diretório `ai/`, de leitura sob demanda. Ele contém quatro *skills* (`laravel-generate`, `livewire` com a política de segurança completa, `bulma-ui` e `asaas`), comandos equivalentes para outras ferramentas, regras no formato do Cursor e modelos de código para controladores, componentes Livewire, modelos, *requests*, testes e marcação Bulma. O `AGENTS.md` anuncia esses arquivos em uma tabela que indica quando cada um deve ser lido; nenhuma ferramenta os carrega automaticamente.

A terceira camada é a configuração de ferramentas. O `opencode.json` lista o `AGENTS.md` como instrução e configura dois servidores de linguagem, o `intelephense` e o `laravel-lsp`, para arquivos PHP e Blade; o `.claude/settings.json` habilita o *plugin* de LSP para PHP no Claude Code (Listagem 2). A intenção declarada no `AGENTS.md` é reduzir o consumo de *tokens*: "prefira o LSP a leituras completas" de arquivos.

Dois detalhes da aplicação do nível `full` merecem registro. O `opencode.json` original

```
## Test-driven, always
Write code test-first: the smallest failing test (red) -> the laziest
code that passes (green) -> refactor while green. **No production code
without a test; everything implemented ships with a test.** [...]

## Laravel conventions
- **Migrations:** always create with ‘php artisan make:migration‘ [...]
  always include a reversible ‘down()‘.
- **Validation:** Livewire components validate with ‘#[Validate]‘ [...]
  **Authorization** always goes through policies/gates, never through
     the UI.
- **Domain writes go through ‘app/Actions/*‘** [...] Don’t add service
    or
  repository layers.
- Set ‘$fillable‘/‘$guarded‘ deliberately (mass-assignment safety).

## Livewire (security-first)
**Every Livewire interaction is an HTTP request and the client controls
everything it can see** [...] Tests come first and must include an
unauthorized-call test and a tamper test for each action.
```

Listagem 1 – Trechos normativos do `AGENTS.md` do *harness* v1 (regra de testes, convenções Laravel e política Livewire)

```
// opencode.json
{ "instructions": ["AGENTS.md"],
  "lsp": {
    "php intelephense": { "command": ["intelephense", "--stdio"],
                          "extensions": [".php", ".blade.php"], ... },
    "laravel-lsp":      { "command": ["laravel-lsp"],
                          "extensions": [".php", ".blade.php"], ... } }
                             }

// .claude/settings.json
{ "env": { "ENABLE_LSP_TOOL": "1" },
  "enabledPlugins": { "php-lsp@claude-plugins-official": true } }
```

Listagem 2 – Configuração de ferramentas do nível `full`: `opencode.json` (abreviado) e `.claude/settings.json`

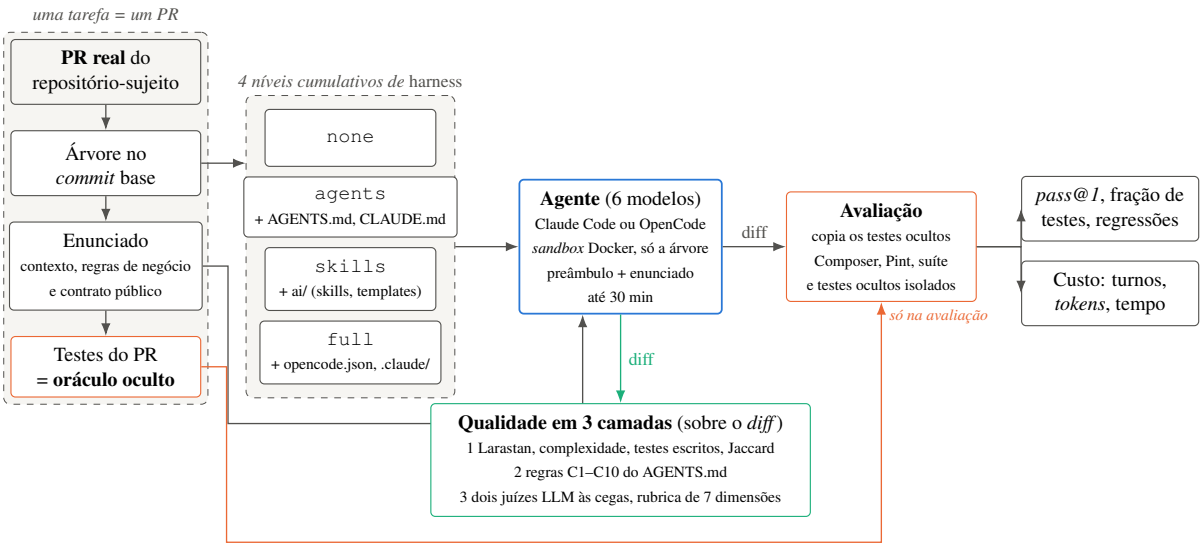
aponta o modelo padrão para o Muse Spark; o orquestrador sobrescreve esse campo com o modelo sob teste e confere, nos eventos da sessão, que nenhum outro modelo respondeu. O *plugin* de LSP do Claude Code só carrega quando instalado no diretório do usuário; o orquestrador o instala em todas as execuções, de modo que o ambiente é idêntico entre níveis e o *plugin* só é ativado onde a árvore o habilita. No OpenCode, o `laravel-lsp` foi instalado no *sandbox* na mesma versão usada pelo repositório-sujeito.

O *BENCHMARK*: TAREFAS, ORÁCULO E AVALIAÇÃO

A Figura 2 resume o desenho: cada PR real do repositório-sujeito gera uma tarefa (árvore base, enunciado e testes ocultos), a árvore recebe um dos quatro níveis de *harness*, um agente

trabalha nela em *sandbox*, e o *diff* resultante é avaliado pelos testes ocultos e, depois, pelas três camadas de qualidade.

Figura 2 – Desenho experimental: de um PR real à avaliação por testes ocultos e por qualidade de código



Fonte: elaborado pelo autor (2026).

As 14 tarefas (Apêndice A) cobrem seis categorias da aplicação: operações (alertas no Discord e canais de *log*), autenticação, multilocação (entidades, associações e convites), domínio (filas, emissão e chamada de senhas), interface Livewire e API. Pelo número de arquivos de produção da solução de referência, cinco são pequenas (até 10 arquivos), sete são médias (11 a 25) e duas são grandes (mais de 25). Duas tarefas vêm de *branches* não mescladas e são consideradas mantidas fora (*held-out*). As soluções de referência variam de 6 a 38 arquivos alterados e de 103 a 2 013 linhas inseridas. Cinco das 14 tarefas são subtarefas de dois PRs compostos: T03a e T03b compartilham o PR de autenticação, e T04a, T04b e T04c compartilham o PR de entidades, sendo que T04b e T04c incluem os testes de T04a. Essas subtarefas não são observações independentes, o que é considerado nas ameaças à validade.

Cada enunciado tem três partes: o contexto do produto, as regras de negócio e um contrato com os nomes de que os testes ocultos dependem. O Quadro 2 mostra as regras centrais do enunciado de T07, a tarefa usada como exemplo na subseção 4.5. Os enunciados não mencionam o *harness* nem dizem ao agente que ele será avaliado por testes. Todos os agentes recebem, antes do enunciado, o mesmo preâmbulo não interativo (Listagem 3), que não fala de testes, de *harness* nem de ferramentas.

O oráculo é oculto: os arquivos de teste que o PR adicionou ou modificou são copiados da solução de referência apenas na avaliação, sobrescrevendo qualquer arquivo de mesmo nome que o agente tenha escrito. Em cinco tarefas (T02, T05, T08, T09 e T11) a solução de referência precisou alterar testes preexistentes, por exemplo, ao mudar o formato dos códigos de senha; esses arquivos fazem parte do oráculo e a sua edição pelo agente não conta como violação, enquanto a edição de qualquer outro teste preexistente conta. A árvore entregue ao agente é um

Quadro 2 – Trecho das regras do enunciado de T07 (chamada da próxima senha), traduzido do original em inglês

Uma senha é **normal** quando `priority = 0` e **prioritária** quando `priority > 0`. 1. Só senhas `waiting` da fila dada são candidatas; uma senha já chamada nunca é chamada de novo. 2. A ordenação é determinística: maior `priority` primeiro, depois `issued_at` mais antigo, depois menor `id`. 3. Modos: **any** (ordenação completa mais a regra de justiça); **priority_only** (só prioritárias, justiça ignorada); **normal_only** (só normais, mais antiga primeiro). 4. **Justiça (só no modo any):** uma senha normal não pode esperar atrás de mais de 3 chamadas prioritárias consecutivas. Olhe as últimas 3 senhas chamadas nesta fila (mais recente primeiro por `called_at`, depois por `id`): se forem 3, todas prioritárias, e houver ao menos uma normal esperando, a chamada fica restrita às normais. Chamadas nos outros modos contam nesse histórico. Uma chamada normal zera a sequência. 5. Chamar de uma fila inativa é rejeitado e não muda nada. Leia o estado da fila no banco, dentro da transação (trave a linha da fila), e não da instância passada. 6. Se nenhuma senha é elegível para o modo pedido, lance uma exceção dedicada. 7. Exatamente uma vez: dentro de uma transação, trave a linha da fila, escolha a candidata e trave-a antes de marcá-la como chamada.

Fonte: adaptado de `benchmark/tasks/T07-call-next-ticket.yaml` (2026).

```
You are working autonomously in a non-interactive session: nobody will
answer questions, review a plan or confirm anything. Do not ask for
clarification or permission. When something is ambiguous, make a
reasonable decision consistent with the task and the existing code, and
keep going until the task is fully implemented.

The task:
```

Listagem 3 – Preâmbulo não interativo anteposto a todo enunciado (texto exato enviado aos agentes)

repositório Git novo, com um único *commit* de base, de modo que a solução não é alcançável pela história, por outras *branches* ou por *tags*.

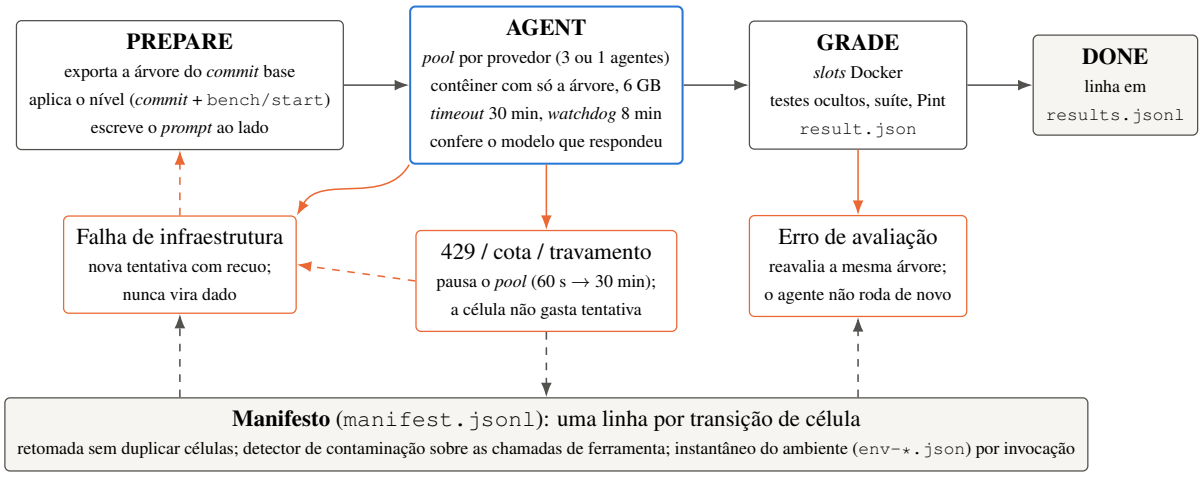
A avaliação, em ordem, adiciona tudo o que o agente deixou (para que arquivos não rastreados contem), registra o *diff* e as violações, copia os testes ocultos, compila os recursos de *front-end* em um contêiner Node e, na imagem de avaliação, instala as dependências com Composer, executa o Pint, a suíte completa e os testes ocultos isoladamente, com SQLite em memória. Se o agente alterou o `composer.json`, as dependências são atualizadas em vez de instaladas, o que exige acesso à rede. Um agente nulo (que não altera nada) e um agente oráculo (que aplica a solução de referência) validaram o avaliador: o oráculo passou em 14 de 14 tarefas e o nulo em 0 de 14. A suíte completa tem dois testes que já falham na árvore exportada, por depender de um diretório excluído da exportação, e essa linha de base é descontada das regressões.

O ORQUESTRADOR

O orquestrador executa a matriz modelo $\times$ tarefa $\times$ nível em três estágios por célula (Figura 3): preparação da árvore e do *prompt*, execução do agente em um *pool* por provedor e avaliação em um número fixo de *slots* de Docker. Ele foi escrito em Python, com a biblioteca padrão, e seu estado é um manifesto em linhas JSON, com uma linha por transição de célula,

que também é a fonte da retomada: a mesma invocação, repetida, pula as células concluídas e continua as pendentes sem duplicar nenhuma.

Figura 3 – Estágios do orquestrador, tratamento de falhas e papel do manifesto



Fonte: elaborado pelo autor (2026).

Isolamento. Durante o desenvolvimento verificou-se que, sem *sandbox*, um agente com permissões automáticas alcançaria, a partir da árvore de trabalho, os enunciados com os arquivos da solução, os *patches* de outras execuções e o próprio repositório-sujeito. Por isso, cada agente roda em um contêiner Docker com apenas a sua árvore montada e nenhum outro caminho do *host*, com acesso completo à internet e com limite de 6 GB de memória, introduzido depois que um contêiner levou o *host* ao limite de memória e foi morto pelo sistema. O comando do projeto para executar testes dentro de contêineres é substituído por um *shim* que executa o comando no próprio *sandbox*, que contém PHP e Composer; assim, o agente pode rodar a suíte como o `AGENTS.md` instrui, em todos os níveis. As credenciais dos provedores nunca ficam no repositório: são exportadas para um diretório temporário por execução e apagadas ao fim.

Detecção de contaminação. Depois de cada execução, as entradas das chamadas de ferramenta são varridas em busca de caminhos fora da árvore e de referências ao repositório-sujeito. Execuções suspeitas são excluídas do *pass@1* e mantidas em disco para auditoria. Durante a rodada, todos os sinais do detector revelaram-se falsos positivos (um *link* para um PR escrito no `CHANGELOG.md` e uma navegação relativa dentro de `vendor/`) e as regras foram corrigidas; o veredito passou a ser recalculado a partir do *log* bruto completo. Tentativas de acessar o repositório-sujeito no GitHub passaram a ser contadas separadamente, porque o repositório é privado e o *sandbox* não tem credencial, de modo que elas não podem vazar nada.

Limites de taxa e falhas. A regra geral é que só o comportamento do modelo vira dado. Um limite de taxa (HTTP 429, cota ou janela de uso) pausa o *pool* do provedor, de 60 segundos até 30 minutos, sem consumir tentativa; outras falhas de infraestrutura são refeitas com recuo; um *timeout* de 30 minutos é dado, com a árvore avaliada como ficou; e uma falha na avaliação leva a uma nova avaliação da mesma árvore, sem rodar o agente de novo. A saída malformada do modelo local, que o servidor rejeita, foi classificada como falha do modelo, para

não enviesar o gpt-oss para cima ao repetir apenas as tentativas em que ele acertou o formato. O Quadro 7 (Apêndice D) registra os incidentes da rodada e as decisões que eles provocaram, todas tomadas sem olhar o resultado das células afetadas. **Reprodutibilidade.** Cada invocação grava um instantâneo do ambiente com as versões do OpenCode, do Claude Code, do Docker e do llama.cpp, os identificadores das imagens, o *commit* do código do orquestrador, o *hash* da matriz e a semente. Cada execução grava o *hash* do preâmbulo, o texto exato do *prompt* enviado, a versão da CLI, o esforço, o *sandbox* e os modelos vistos na sessão. Testes de aceitação verificaram que a execução sequencial e a paralela produzem o mesmo resultado, que a retomada não duplica células e que o 429 pausa o *pool*.

RESULTADOS: *PASS@1* E CUSTO

A Tabela 1 resume os modelos e parâmetros, e a Tabela 2 apresenta o *pass@1* por modelo e nível, com intervalos de confiança de Wilson a 95%.

Tabela 1 – Modelos avaliados, *runners* e parâmetros

Modelo	**Runner**	**Parâmetros**	**Concorrência**
Claude Sonnet 5.5	Claude Code 2.1.312	esforço `high`	3
Claude Haiku 5.5	Claude Code 2.1.312	esforço `high`	3
Muse Spark 1.3	OpenCode v2.2.0 (Go)	padrão do provedor	3
DeepSeek v4.1 flash	OpenCode v2.2.0 (Go)	padrão do provedor	3
Beam (free)	OpenCode v2.2.0 (Go/Zen)	padrão do provedor	3
gpt-oss-20b (local)	OpenCode v2.2.0 + llama.cpp	MXFP4, 128k, temp. 1,0	1

Fonte: elaborado pelo autor (2026).

Tabela 2 – *Pass@1* por modelo e nível de *harness* (tarefas resolvidas em 14; IC 95% de Wilson entre colchetes)

Modelo	`none`	`agents`	`skills`	`full`	**Total**
Sonnet 5.5	14 [0,78; 1,00]	14 [0,78; 1,00]	14 [0,78; 1,00]	13 [0,69; 0,99]	55/56 (0,98)
DeepSeek v4.1 flash	14 [0,78; 1,00]	14 [0,78; 1,00]	14 [0,78; 1,00]	14 [0,78; 1,00]	56/56 (1,00)
Muse Spark 1.3	13 [0,69; 0,99]	13 [0,69; 0,99]	14 [0,78; 1,00]	13 [0,69; 0,99]	53/56 (0,95)
Haiku 5.5	11 [0,52; 0,92]	11 [0,52; 0,92]	11 [0,52; 0,92]	11 [0,52; 0,92]	44/56 (0,79)
gpt-oss-20b	2 [0,04; 0,40]	1 [0,01; 0,31]	1 [0,01; 0,31]	2 [0,04; 0,40]	6/56 (0,11)
Beam (free)[a]	10/10	7/7	5/5	7/7	29/29 (1,00)

Fonte: elaborado pelo autor (2026). [a] Beam incompleto: 29 das 56 células (23 pelo OpenCode Go e 6 pelo OpenCode Zen); as células concluídas não são uma amostra aleatória das tarefas e ficam fora das comparações entre níveis.

Efeito de teto. Quatro dos seis modelos resolveram 95% ou mais das tarefas já no nível `none`: Sonnet 5.5 (55/56 no total), DeepSeek v4.1 flash (56/56), Muse Spark 1.3 (53/56) e Beam (29/29 nas células que rodaram). Com o teto tão próximo, não há espaço para o *harness* melhorar o *pass@1* desses modelos. Mesmo quando falham, os modelos fortes falham por um único teste (Tabela 3): a fração média de testes ocultos aprovados fica em 0,998 ou mais para eles e em 0,993 para o Haiku 5.5 em todos os níveis, enquanto o gpt-oss-20b aprova cerca de um quarto dos testes.

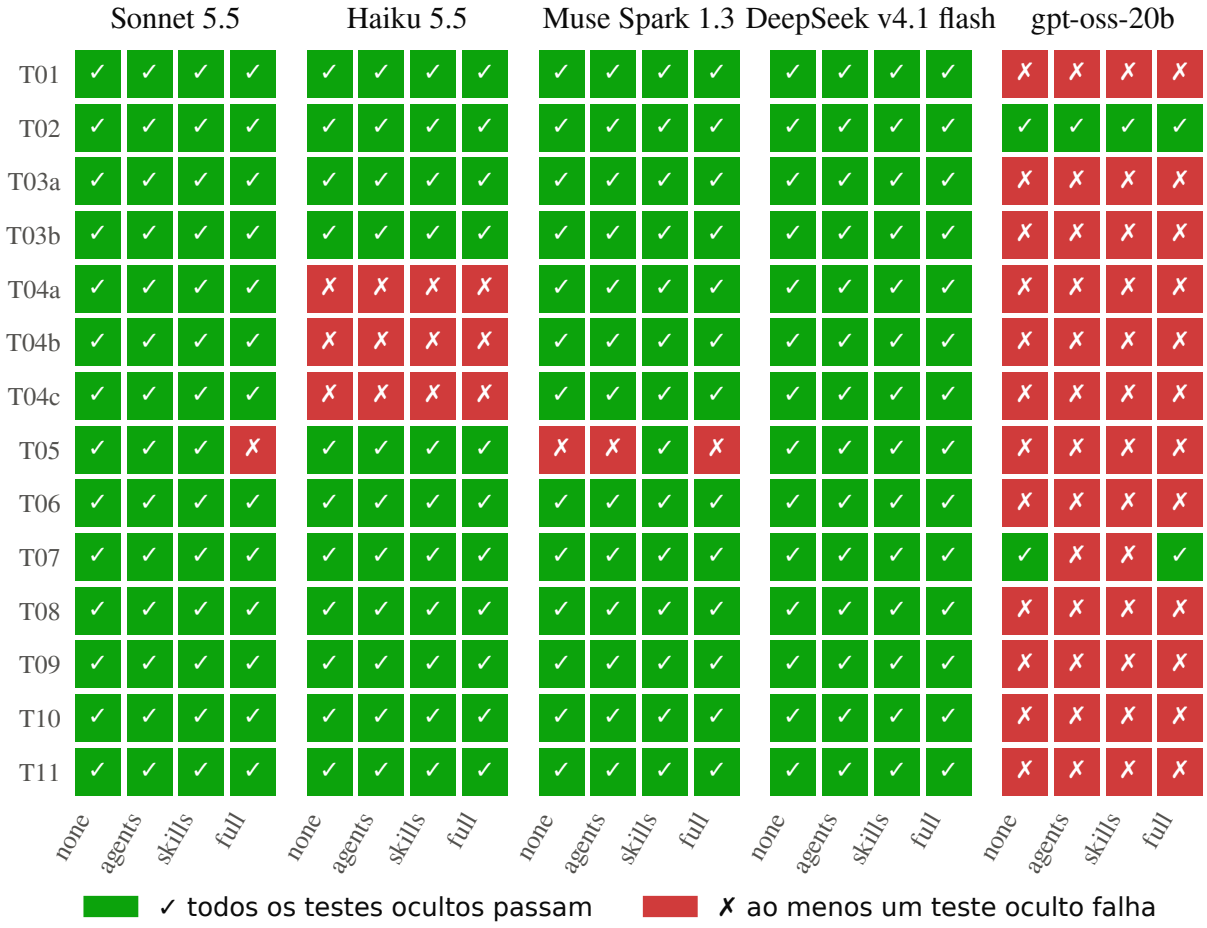
Tabela 3 – Fração média de testes ocultos aprovados por modelo e nível

Modelo	`none`	`agents`	`skills`	`full`
Sonnet 5.5, DeepSeek v4.1 flash, Muse Spark 1.3, Beam	$\geq 0{,}998$	$\geq 0{,}998$	$\geq 0{,}998$	$\geq 0{,}998$
Haiku 5.5	0,993	0,993	0,993	0,993
gpt-oss-20b	0,250	0,244	0,203	0,203

Fonte: elaborado pelo autor (2026).

Nenhum efeito do *harness* sobre o *pass@1*. A Tabela 4 mostra, para cada modelo e nível, o número de tarefas em que apenas o nível passou ($+$) e em que apenas o `none` passou ($-$). Nos 70 pares modelo $\times$ tarefa de cada nível (5 modelos completos $\times$ 14 tarefas), houve no máximo uma discordância para cada lado. As diferenças agregadas foram de $-1{,}4$ pontos percentuais para `agents`, 0,0 para `skills` e $-1{,}4$ para `full`, com $p = 1{,}0$ no teste exato de McNemar nos três casos. Em 5 modelos $\times$ 3 níveis houve apenas quatro discordâncias, nas duas direções. A Figura 4 torna o padrão visível: as colunas de um mesmo modelo são quase idênticas entre níveis, e as falhas se repetem na mesma tarefa em todos os níveis (Haiku em T04a, T04b e T04c; Muse em T05; gpt-oss em quase tudo). O resultado é dominado pelo modelo e pela tarefa, não pelo nível.

Figura 4 – Resultado dos testes ocultos por tarefa, modelo e nível de *harness* (cinco modelos com a matriz completa)



Fonte: elaborado pelo autor (2026).

Tabela 4 – Efeito de cada nível contra `none` no *pass@1*, pareado por tarefa (+: só o nível passou; −: só o `none` passou)

Modelo	`agents` (+/−)	`skills` (+/−)	`full` (+/−)
Sonnet 5.5	0/0	0/0	0/1 (T05)
DeepSeek v4.1 flash	0/0	0/0	0/0
Muse Spark 1.3	0/0	1/0 (T05)	0/0
Haiku 5.5	0/0	0/0	0/0
gpt-oss-20b	0/1 (T07)	0/1 (T07)	0/0
Agregado (70 pares)	0/1; $\Delta = -1{,}4$ pp; $p = 1{,}0$	1/1; $\Delta = 0{,}0$ pp; $p = 1{,}0$	0/1; $\Delta = -1{,}4$ pp; $p = 1{,}0$

Fonte: elaborado pelo autor (2026).

Os dois modelos abaixo do teto não reagiram ao *harness*. O Haiku 5.5 resolveu 11 de 14 tarefas em todos os níveis e o gpt-oss-20b 1 ou 2; em ambos, as falhas são sistemáticas e independem do nível (Quadro 3). No caso do Haiku, trata-se de uma interpretação estável do modelo, uma proteção ao dono da entidade que o enunciado não pede, e que nenhum nível de *harness* corrigiu. No caso do gpt-oss, a métrica contínua sugere uma piora leve nos níveis `skills` e `full` (de 0,250 para 0,203), sem significância com $n = 14$.

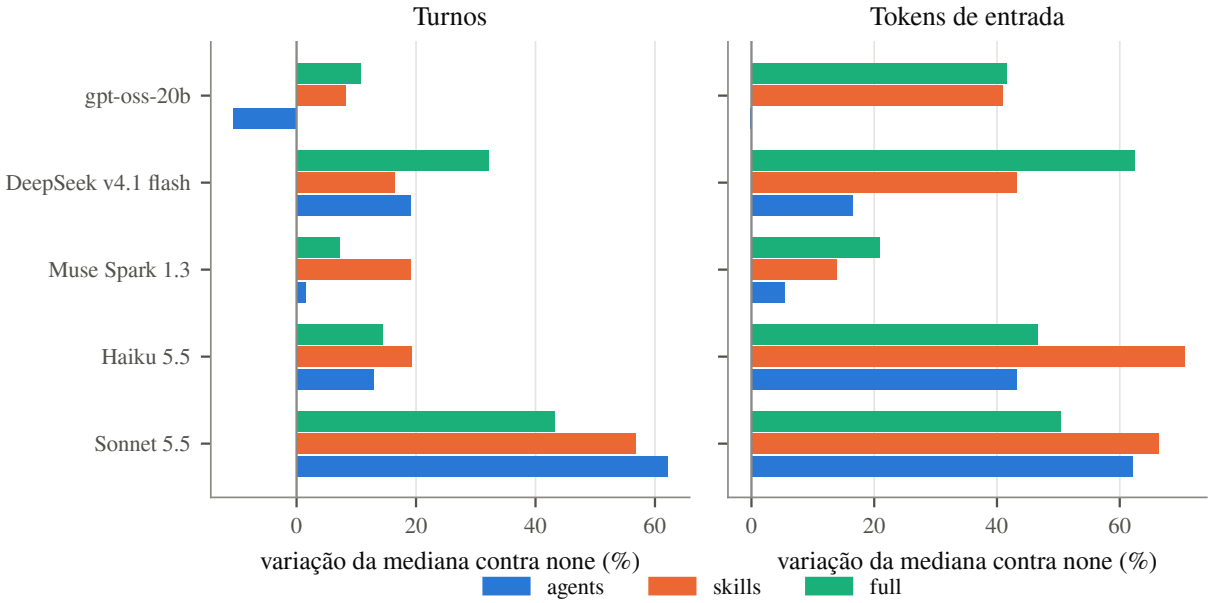
Quadro 3 – Falhas sistemáticas observadas na rodada reduzida

Modelo	**Tarefas**	**O que falha**	**Causa e relação com o nível**
Haiku 5.5	T04a, T04b, T04c	Um único teste de política: `assignRole(admin, ownerMembership)` deveria ser `true`	O enunciado diz que dono e administrador podem atribuir papéis; o modelo impede o administrador de alterar o papel do dono. Igual nas 12 células, em todos os níveis.
Muse Spark 1.3	T05	Um teste do componente de membros, variável entre execuções	Falha em `none`, `agents` e `full`; passa em `skills`. Única discordância positiva da rodada.
Sonnet 5.5	T05	Um teste em `full`	Única falha do modelo em 56 células.
gpt-oss-20b	12 das 14 tarefas	24 de 56 execuções terminam com saída que o servidor não consegue interpretar; 35 deixam regressões além da base (média de 5,5 testes); Pint passa em 3	Limitação de capacidade e de formato de saída do conjunto modelo mais servidor; nenhum nível muda o quadro.

Fonte: elaborado pelo autor (2026).

O *harness* aumenta o custo. A Figura 5 mostra a variação relativa ao `none`; as medianas absolutas estão na Tabela 12, no Apêndice D. Com *harness*, o Sonnet 5.5 gasta 44% a 67% mais turnos e 50% a 66% mais *tokens* de entrada; o Haiku 5.5, 13% a 19% e 43% a 71%; o DeepSeek e o Muse, 17% a 62% e 5% a 21% em *tokens* de entrada. O gpt-oss, que raramente abre os arquivos do *harness*, quase não muda. Os três níveis com *harness* custam de forma parecida entre si: a maior parte do acréscimo vem da primeira camada. Em totais, a rodada consumiu 3,7 horas de agente no Sonnet (US$ 38,7 a preço de lista, pela estimativa da própria CLI), 7,0 horas no Haiku (US$ 27,3), 6,1 no Muse, 8,1 no DeepSeek e 3,0 no gpt-oss; o Haiku gerou cerca de três vezes mais *tokens* de saída que o Sonnet, com *pass@1* menor. O tempo de parede não é comparável entre modelos, pois depende do provedor, do *hardware* e da concorrência; turnos e *tokens* são a medida comparável.

Figura 5 – Variação da mediana de turnos e de *tokens* de entrada de cada nível em relação ao `none`, por modelo



Fonte: elaborado pelo autor (2026). Tokens de entrada incluem leitura de *cache*.

Comportamento. Houve um único *timeout* válido (DeepSeek em T05 `none`, que trabalhou os 30 minutos e passou mesmo assim) e nenhuma contaminação. A Tabela 5 resume o que os agentes deixaram além do *pass@1*: nenhum editou arquivos do *harness* nem introduziu saída Blade sem escape; as regressões concentram-se no gpt-oss; e Sonnet e Haiku são os que mais alteram testes preexistentes. Três execuções tentaram buscar o repositório-sujeito no GitHub (uma leitura de arquivo bruto, uma consulta à API e um `git ls-remote`), todas do DeepSeek e todas no nível `none`; todas falharam com erro ou 404. Com $n = 3$ o fato é anedótico, mas é coerente com a ideia de que o *harness* dá ao agente contexto suficiente para não procurar fora.

Tabela 5 – Comportamento além do *pass@1*: execuções com regressão, aprovação no Pint e violações, por modelo

Modelo	Com regressão	Pint ok	Alterou testes	Editou o harness	Blade sem escape
Sonnet 5.5	3/56	43/56	11/56	0	0
Haiku 5.5	5/56	44/56	9/56	0	0
Muse Spark 1.3	1/56	40/56	0/56	0	0
DeepSeek v4.1 flash	2/56	45/56	3/56	0	0
Beam (free)	0/29	19/29	2/29	0	0
gpt-oss-20b	35/56	3/56	0/56	0	0

Fonte: elaborado pelo autor (2026). “Alterou testes” conta execuções em que o agente editou testes preexistentes da árvore; os testes ocultos nunca são visíveis para ele.

RESULTADOS: QUALIDADE DE CÓDIGO

A análise de qualidade cobriu as 309 execuções avaliadas da rodada (56 de cada um dos cinco modelos completos e 29 do Beam), com as três camadas do Quadro 1 aplicadas a todas, em 836 revisões de juízes. Cada mudança recebeu primeiro dois juízes de famílias distintas da do modelo que a escreveu, em pares fixos por modelo avaliado: Muse e DeepSeek para Sonnet e

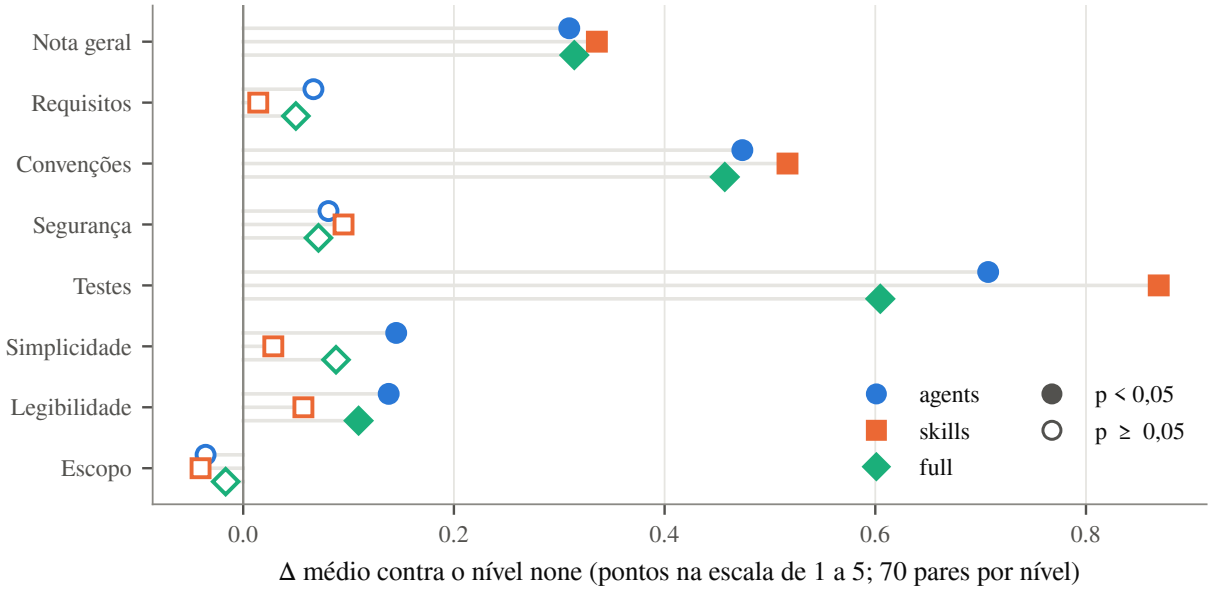
Haiku; Opus e DeepSeek para o Muse; Opus e Muse para o DeepSeek; rotação balanceada para gpt-oss e Beam (615 revisões; 306 execuções com as duas e três com uma, por erro do segundo juiz). Em seguida, o juiz da mesma família revisou as mudanças da própria família: Opus as de Sonnet e Haiku, Muse as do Muse e DeepSeek as do DeepSeek (221 revisões; três perdidas por erro do DeepSeek). Assim, 218 execuções têm três revisões e 91 têm duas; gpt-oss e Beam não têm juiz da própria família. Em todas as tabelas desta seção, a nota de uma execução é a média de todos os seus juízes; a razão dessa escolha e a comparação entre o juiz da família e os de fora estão no fim da seção. Cada juiz trabalhou em um espaço isolado com o enunciado, o *diff* sem os arquivos do *harness*, as seções normativas do `AGENTS.md` como padrão de revisão (o mesmo para todos os níveis), a rubrica e o esquema de saída; uma verificação recusa qualquer espaço que contenha o identificador da execução, o modelo, o nível ou o resultado dos testes.

Efeito por nível. A Tabela 6 resume o efeito de cada nível contra o `none`, pareado por tarefa, nos 70 pares dos cinco modelos completos. O *harness* elevou a nota geral dos juízes em 0,31 a 0,34 ponto (em uma escala de 1 a 5; $p < 0{,}001$), a dimensão de convenções em 0,46 a 0,52 ($p < 0{,}001$) e a dimensão de testes em 0,60 a 0,87 ($p < 0{,}001$). Na camada estática, o número de casos de teste novos subiu 10 a 11 por execução ($p < 0{,}001$), o escore de convenções subiu 0,07 a 0,10 ($p \leq 0{,}001$), os erros novos do Larastan caíram 0,5 a 0,6 por execução (p entre 0,015 e 0,057) e a complexidade média caiu 0,16 a 0,19 (p entre 0,001 e 0,051). Requisitos, segurança e escopo não mudaram (p entre 0,05 e 0,86), nem a distância da solução de referência (Jaccard, $p \geq 0{,}17$). A legibilidade subiu pouco (0,06 a 0,14; p entre 0,003 e 0,245) e a simplicidade só no nível `agents` (0,15; $p = 0{,}015$). A Figura 6 mostra esse contraste entre as dimensões: três sobem em todos os níveis, cinco ficam junto do zero. O nível `skills` tem os maiores efeitos em convenções e testes, mas as diferenças entre `agents`, `skills` e `full` são pequenas diante dos efeitos de cada um contra o `none`: a maior parte do ganho vem da primeira camada.

Efeito por modelo. A Tabela 7 e a Figura 7 mostram a nota geral dos juízes e o número de casos de teste novos por modelo e nível. O efeito é maior em quem parte mais baixo. O DeepSeek v4.1 flash é quem mais ganha: a nota geral sobe de 3,21 para 3,89 a 3,99, a nota de testes de 2,74 para 4,36 a 4,46 e os casos de teste de 8,0 para 27,8 a 32,1 por execução. O Muse Spark 1.3 vem em seguida (nota geral de 2,57 para 3,07 a 3,36). Sonnet e Haiku ganham pouco na nota geral, porque já partem de convenções e testes altos, mas o Sonnet dobra os casos de teste escritos (de 16,7 para cerca de 34). Os dois modelos mais fracos não respondem: o gpt-oss-20b e o Beam ficam com nota de testes de 1,0 em quase todos os níveis e nota geral abaixo de 2,7. Neles, as regras do `AGENTS.md` que mandam escrever testes simplesmente não são seguidas.

Volume de código. A Tabela 8 e a Figura 8 separam as linhas inseridas por execução em código de produção, testes e outros arquivos (documentação, traduções e configuração do projeto), com a classificação da camada 1 e sem os arquivos do *harness*. O resultado é nítido: o *harness* não muda o tamanho da solução e acrescenta testes a ela. Nos cinco modelos completos, as linhas de produção ficam em 470 a 475 por execução em todos os níveis (Δ entre $-0{,}8$ e $+4{,}0$

Figura 6 – Efeito de cada nível contra `none` nas dimensões dos juízes (marcadores cheios: $p < 0{,}05$ no Wilcoxon exato)



Fonte: elaborado pelo autor (2026).

Tabela 6 – Efeito de cada nível contra `none` nas métricas de qualidade (Δ = média das diferenças pareadas por tarefa; Wilcoxon exato; 70 pares por nível, 63 a 70 nas métricas com dados ausentes)

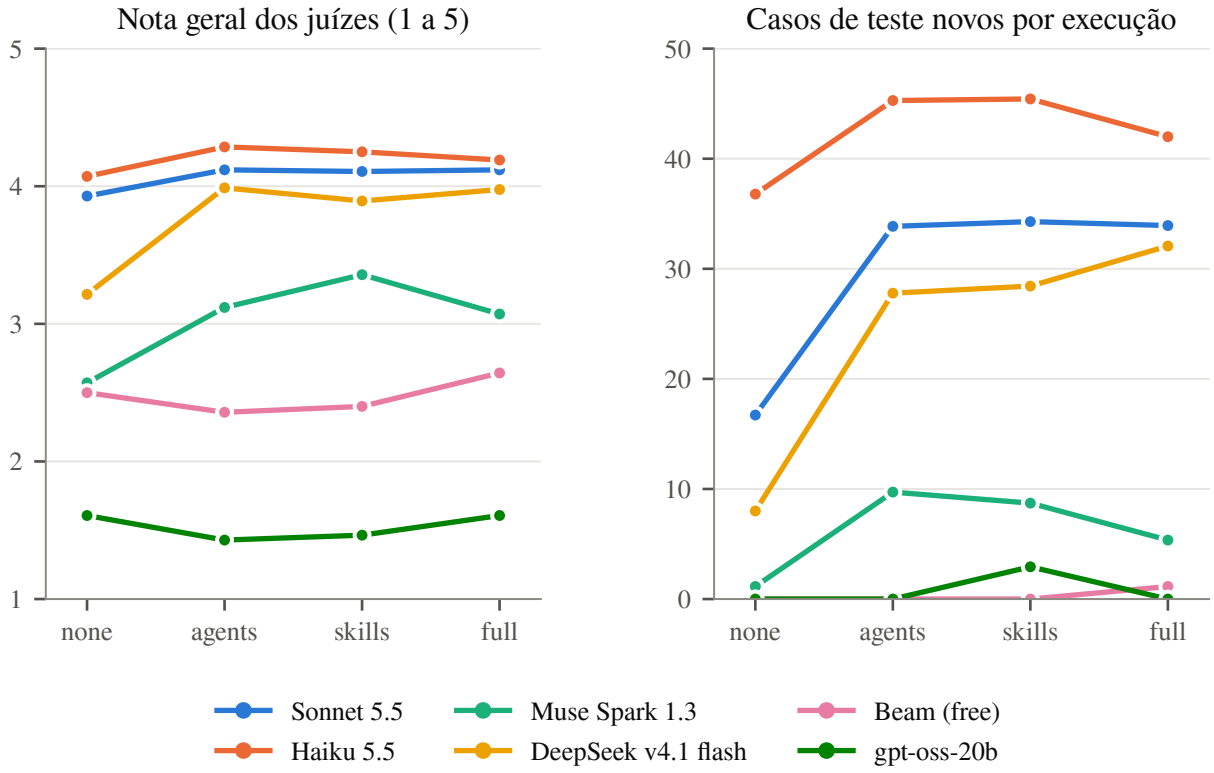
	`agents`		`skills`		`full`	
Métrica	Δ	*p*	Δ	*p*	Δ	*p*
Camada 1 e 2 (medidas objetivas)						
Escore de convenções (0 a 1)	+0,07	0,001	+0,10	< 0,001	+0,07	< 0,001
Larastan: erros novos	−0,53	0,034	−0,57	0,057	−0,63	0,015
Complexidade média	−0,17	0,001	−0,19	0,025	−0,16	0,051
Casos de teste novos	+10,80	< 0,001	+11,43	< 0,001	+10,14	< 0,001
Jaccard vs. referência	−0,00	0,265	−0,01	0,245	−0,01	0,169
Camada 3 (juízes, notas de 1 a 5)						
Nota geral	+0,31	< 0,001	+0,34	< 0,001	+0,31	< 0,001
Requisitos	+0,07	0,081	+0,01	0,447	+0,05	0,316
Convenções	+0,47	< 0,001	+0,52	< 0,001	+0,46	< 0,001
Segurança	+0,08	0,177	+0,10	0,114	+0,07	0,052
Testes	+0,71	< 0,001	+0,87	< 0,001	+0,60	< 0,001
Simplicidade	+0,15	0,015	+0,03	0,508	+0,09	0,075
Legibilidade	+0,14	0,003	+0,06	0,245	+0,11	0,015
Escopo	−0,04	0,857	−0,04	0,739	−0,02	0,844

Fonte: elaborado pelo autor (2026). O Beam, incompleto, não entra no agregado. Camada 3: média de todos os juízes da execução. *p*-valores sem correção para comparações múltiplas (39 comparações).

linha; $p \geq 0{,}79$), enquanto as linhas de teste sobem de 183 para 317 a 342 (Δ de +134 a +159; $p < 0{,}001$) e a parcela de testes no código passa de 22% para 34% a 38% ($p < 0{,}001$). O padrão por modelo repete o das notas: o Sonnet vai de 36% para 50% de testes, o DeepSeek de 22% para 49% a 50%, o Muse de 2% para 15% a 27%; o Haiku, que já escrevia tantas linhas de teste quanto de produção, vai de 52% para 55% a 57%; o gpt-oss e o Beam ficam em zero em quase todos os níveis. Os demais arquivos não mudam (55 a 59 linhas por execução).

Um exemplo. O caso do Muse Spark na tarefa T07 (Quadro 2) ilustra o padrão. Nos

Figura 7 – Nota geral dos juízes e casos de teste novos por execução, por modelo e nível de *harness*



Fonte: elaborado pelo autor (2026).

Tabela 7 – Nota geral dos juízes (média de todos os juízes da execução, 1 a 5) e casos de teste novos por execução, por modelo e nível

	Nota geral dos juízes				Casos de teste novos			
Modelo	`none`	`agents`	`skills`	`full`	`none`	`agents`	`skills`	`full`
Sonnet 5.5	3,93	4,12	4,11	4,12	16,7	33,9	34,3	33,9
Haiku 5.5	4,07	4,29	4,25	4,19	36,8	45,3	45,4	42,0
Muse Spark 1.3	2,57	3,12	3,36	3,07	1,1	9,7	8,7	5,4
DeepSeek v4.1 flash	3,21	3,99	3,89	3,98	8,0	27,8	28,4	32,1
Beam (free)[a]	2,50	2,36	2,40	2,64	0,0	0,0	0,0	1,1
gpt-oss-20b	1,61	1,43	1,46	1,61	0,0	0,0	2,9	0,0

Fonte: elaborado pelo autor (2026). $n = 14$ por célula. [a] Beam: n = 10, 7, 5 e 7.

quatro níveis o modelo passou nos testes ocultos. No nível `none`, o *diff* trouxe a ação de domínio, o *enum* de modos, a exceção e a documentação, e nenhum arquivo de teste; os três juízes deram nota 1 em testes e nota geral 3, 3 e 2. No nível `skills`, o mesmo modelo entregou, além do código, um arquivo de teste com quinze casos cobrindo ordenação e empates, escopo por fila, os três modos e seus caminhos de erro, fila vazia, a regra de justiça e fila inativa; os três juízes deram nota geral 4 e nota 4 ou 5 em testes. A Listagem 4 reproduz, de forma abreviada, as duas revisões do mesmo juiz. O oráculo oculto não distingue as duas entregas; a revisão de código distingue.

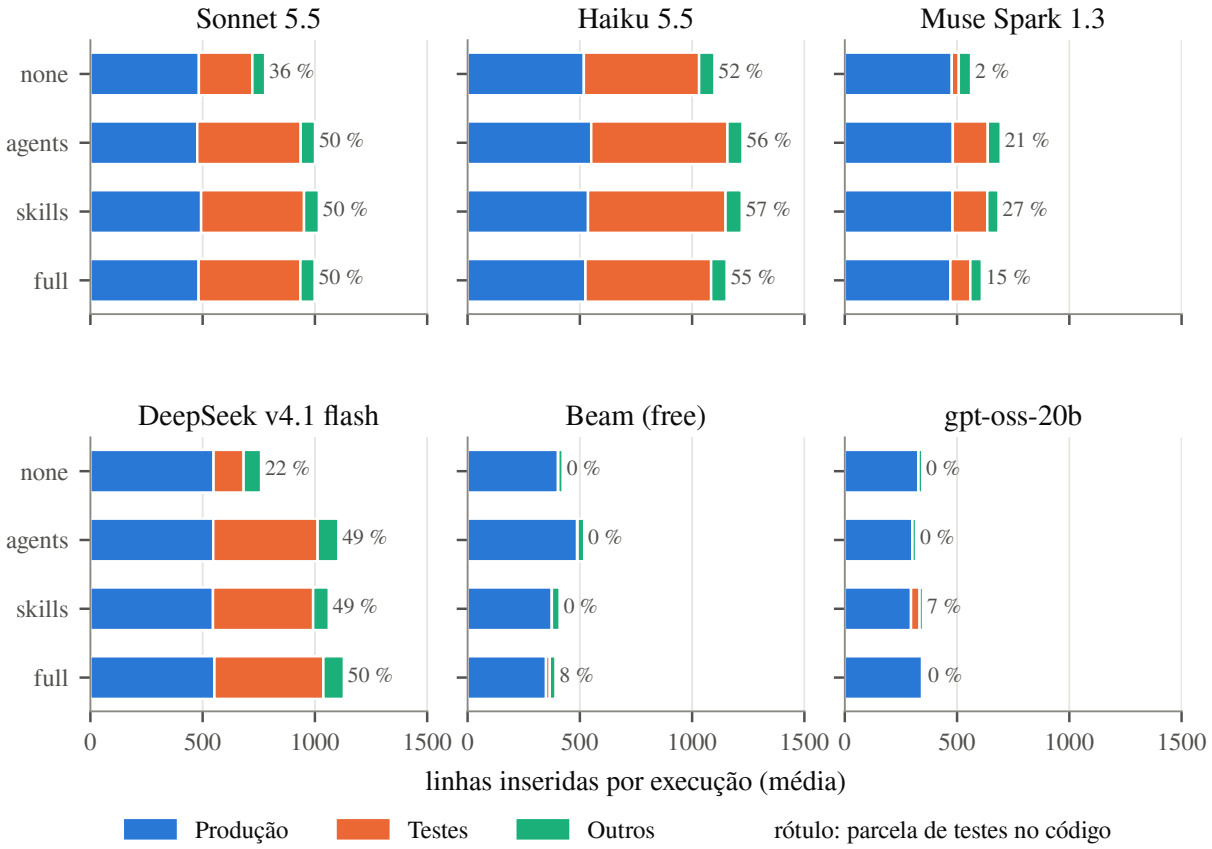
Concordância e validade dos juízes. A Tabela 9 e a Figura 9 mostram a concordância entre as três duplas de juízes na nota geral, em todas as mudanças que os dois avaliaram. O κ ponderado vai de 0,71 (Muse e DeepSeek) a 0,84 (Opus e DeepSeek), o que corresponde a

Tabela 8 – Linhas inseridas por execução (média) em código de produção e em testes, por modelo e nível

	Linhas de produção				Linhas de teste			
Modelo	`none`	`agents`	`skills`	`full`	`none`	`agents`	`skills`	`full`
Sonnet 5.5	483	476	493	482	238	461	458	453
Haiku 5.5	518	551	536	525	513	607	613	560
Muse Spark 1.3	476	481	480	470	31	156	155	89
DeepSeek v4.1 flash	549	547	546	552	134	465	446	485
Beam (free)[a]	403	488	375	349	1	2	0	17
gpt-oss-20b	328	301	295	344	0	0	38	0

Fonte: elaborado pelo autor (2026). Produção: `app/`, `routes/`, `resources/`, `database/` e `config/` em PHP, JavaScript ou CSS; testes: `tests/`. Arquivos do *harness* excluídos; linhas removidas não contadas. $n = 14$ por célula. [a] Beam: n = 10, 7, 5 e 7.

Figura 8 – Linhas inseridas por execução, por categoria, modelo e nível de *harness* (rótulo: parcela de testes no código)



Fonte: elaborado pelo autor (2026).

concordância substancial a quase perfeita na escala de Landis e Koch (1977); o κ médio nas sete dimensões vai de 0,63 a 0,80. Na figura, as discordâncias concentram-se em um ponto da escala, quase sempre com o Muse acima do outro juiz.

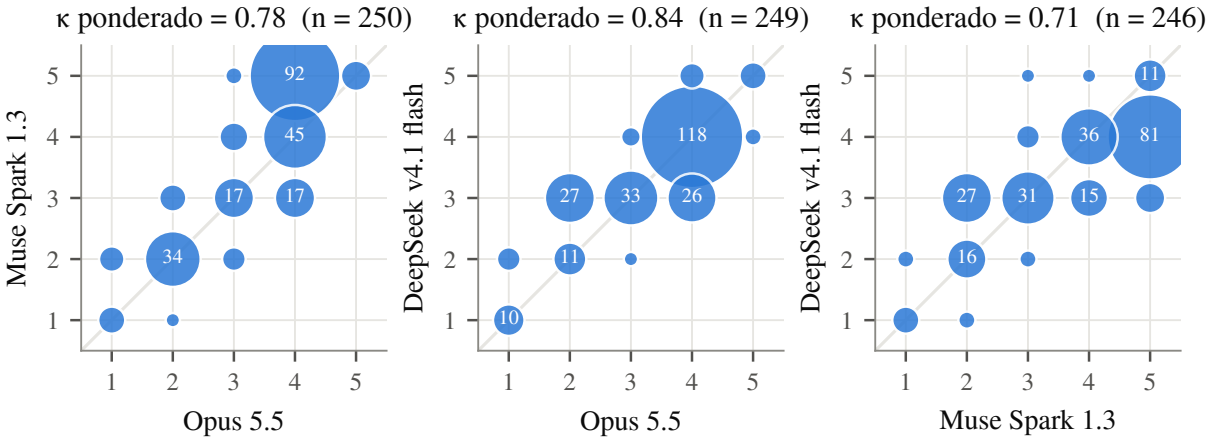
Viés e autopreferência dos juízes. Os juízes têm vieses pequenos e em direções distintas: o Muse é o mais generoso (+0,21 em relação à média dos juízes da mesma execução), o Opus o mais severo ($-0{,}13$) e o DeepSeek fica perto do centro ($-0{,}07$). A generosidade do Muse, porém, não é uniforme: nas mudanças do Sonnet e do Haiku ele dá, em média, 4,32 e 4,68, contra 3,85 e 3,89 do DeepSeek nas mesmas mudanças, enquanto nas do gpt-oss e do Beam fica

```
// T07, nivel none (Muse Spark 1.3)                    hidden_ok: true
{ "scores": { "requirements": 4, "conventions": 2, "security": 5,
              "tests": 1, "simplicity": 4, "readability": 4, "scope": 5
                 },
  "overall": 3,
  "issues": [ { "severity": "blocker", "dimension": "tests",
      "location": "change.diff (no tests/ files)",
      "explanation": "Concurrency-sensitive domain logic ships with no
        tests at all, which breaks the project's mandatory TDD rule." },
    ... ],
  "summary": "... ships with no tests at all for concurrency-critical
    logic, which the project's mandatory TDD convention forbids, so it
    needs changes before merging." }

// T07, nivel skills (Muse Spark 1.3)                  hidden_ok: true
{ "scores": { "requirements": 4, "conventions": 4, "security": 5,
              "tests": 4, "simplicity": 4, "readability": 5, "scope": 5
                 },
  "overall": 4,
  "issues": [ { "severity": "minor", "dimension": "tests",
      "location": "tests/Feature/CallNextTicketTest.php",
      "explanation": "The exactly-once skip path ... has no test ..." },
    ... ],
  "summary": "... applies the 3-call fairness rule only in Any mode ...;
    it also ships thorough Pest tests and the required doc and CHANGELOG
    updates. I'd approve it with nits ..." }
```

Listagem 4 – Revisões do juiz Opus 5.5 para o Muse Spark em T07, nos níveis `none` e `skills` (`review.json` abreviado)

Figura 9 – Nota geral atribuída pelos dois juízes de cada dupla à mesma mudança (área do círculo proporcional ao número de mudanças)



Fonte: elaborado pelo autor (2026).

abaixo do DeepSeek. Se as notas de Sonnet e Haiku dependessem só do par Muse e DeepSeek, o mais leniente dos três pares possíveis, a comparação absoluta entre modelos ficaria distorcida. Por isso o juiz da mesma família foi acrescentado, e a sua inclusão foi avaliada como controle de autopreferência: na mesma mudança, compara-se a nota do juiz da família com a média dos dois de fora (Wilcoxon exato pareado). O Opus deu $-0{,}17$ a Sonnet e Haiku ($p = 0{,}003$; $n = 109$), o Muse $+0{,}04$ ao Muse ($p = 0{,}51$; $n = 56$) e o DeepSeek $-0{,}17$ ao DeepSeek ($p = 0{,}039$; $n = 53$).

Descontada a leniência geral de cada juiz, medida nas mudanças de outras famílias ($-0{,}33$ para o Opus, $+0{,}44$ para o Muse e $-0{,}18$ para o DeepSeek), a autopreferência líquida é de $+0{,}17$ para o Opus, $-0{,}40$ para o Muse e próxima de zero para o DeepSeek: não há autopreferência consistente, e a diferença bruta entre o juiz da família e os de fora é quase toda o viés geral de cada juiz. O efeito da inclusão sobre as médias é pequeno: nas execuções com três revisões, a nota geral do Sonnet passa de 4,10 para 4,08, a do Haiku de 4,28 para 4,19, a do Muse de 3,02 para 3,03 e a do DeepSeek de 3,83 para 3,77. Como o conjunto de juízes é fixo por modelo avaliado, o viés residual é constante entre os níveis de um mesmo modelo e não afeta as comparações pareadas; a comparação de níveis absolutos entre modelos continua a depender de quais juízes avaliam cada modelo e deve ser lida com essa reserva.

Tabela 9 – Concordância entre juízes na nota geral e nas sete dimensões (todas as mudanças avaliadas pelos dois juízes da dupla)

Dupla	n	κ ponderado (geral)	Spearman (geral)	κ médio (7 dimensões)
Opus × Muse	250	0,78	0,81	0,63
Opus × DeepSeek	249	0,84	0,83	0,80
Muse × DeepSeek	246	0,71	0,78	0,73

Fonte: elaborado pelo autor (2026).

A Tabela 10 verifica se as notas medem algo externo a elas. A nota geral correlaciona-se com o acerto nos testes ocultos ($\rho = 0{,}53$), e a dimensão de requisitos um pouco mais ($\rho = 0{,}55$): os juízes, que não veem o resultado dos testes, tendem a dar notas mais altas às mudanças que passam. A dimensão de convenções dos juízes correlaciona-se fortemente com o escore heurístico da camada 2 ($\rho = 0{,}73$), duas medidas independentes do mesmo construto. Em contrapartida, a nota geral não se correlaciona com os erros novos do Larastan ($\rho = -0{,}07$); no piloto, com duas tarefas, essa correlação era de $-0{,}53$, e a divergência é discutida nas ameaças à validade.

Tabela 10 – Validade: correlação de Spearman entre medidas de camadas distintas (todas as execuções)

Medida A	**Medida B**	ρ	n
Juízes: nota geral	Testes ocultos aprovados	0,53	309
Juízes: requisitos	Testes ocultos aprovados	0,55	309
Juízes: convenções	Escore de convenções (camada 2)	0,73	308
Juízes: nota geral	Larastan: erros novos	$-0{,}07$	294
Escore de convenções (camada 2)	Testes ocultos aprovados	0,37	308

Fonte: elaborado pelo autor (2026).

Regras de convenção. Das dez regras heurísticas (Apêndice C), as mais violadas no conjunto foram C4 (escritas de domínio por classes de ação: cumprida em 39 das 148 execuções em que se aplicou), C3 (autorização *server-side* de toda ação Livewire: 36 de 87), C2 (validação com `#[Validate]`: 53 de 106), C8 (`$fillable` ou `$guarded` declarados: 84 de 151) e C1 (código de produção acompanhado de teste: 193 de 308). As regras de escape em Blade (C7), de ausência de camadas de serviço (C6), de interface em Bulma (C9) e de migrações geradas e reversíveis (C5) foram cumpridas em quase todas as execuções em que se aplicaram. No piloto, as 25 violações apontadas foram lidas manualmente contra o código, e todas eram reais (C1 19

de 19, C2 2 de 2, C10 4 de 4); uma correção na rodada completa fez a regra C8 reconhecer a sintaxe de atributo `#[Fillable]`, que antes era contada como violação.

DISCUSSÃO

O *harness* muda como o código é escrito, não se a tarefa é resolvida. Os dois conjuntos de resultados apontam na mesma direção. Pelo oráculo oculto, nenhum nível alterou o *pass@1* de nenhum modelo: o *harness* não deixou o modelo mais capaz de resolver um problema que ele não conseguia resolver; pelos juízes e pela camada estática, todos os níveis elevaram convenções, testes e casos de teste escritos, sem tocar em requisitos, segurança ou escopo. A dissociação é coerente com o que o *harness* contém: o `AGENTS.md` não ensina o domínio nem corrige a leitura de um enunciado, mas prescreve como o código deve ser organizado e exige que cada mudança venha com testes. As linhas inseridas confirmam essa leitura: o código de produção tem o mesmo tamanho em todos os níveis, e o que o *harness* acrescenta são 134 a 159 linhas de teste por execução. O efeito sobre a dimensão de testes é, provavelmente, o mais direto: a regra "nenhum código de produção sem teste" é lida e seguida pelos modelos de capacidade intermediária e alta. O exemplo da T07 mostra o limite do oráculo: um PR sem nenhum teste e um PR com quinze testes passam igualmente nos testes ocultos, mas um revisor humano não os aprovaria da mesma forma.

Escala explica o *pass@1*; o *harness* explica parte da qualidade. Entre modelos, o *pass@1* vai de 11% a 100%; entre níveis, para um mesmo modelo, a variação é de no máximo 7 pontos percentuais. A escolha do modelo explica quase toda a variação em acerto, e o *harness* não compensa a diferença: o Haiku 5.5 não alcança o Sonnet 5.5 em nenhum nível, e o gpt-oss-20b não sai do patamar de 1 a 2 tarefas em 14. Na qualidade, o quadro é diferente. O DeepSeek v4.1 flash com *harness* recebe dos juízes uma nota geral (3,89 a 3,99) comparável à do Sonnet 5.5 sem *harness* (3,93), ambas com três juízes, e escreve um número de testes (28 a 32 por execução) semelhante ao do Sonnet com *harness* (34). Nesse sentido estrito, e apenas nessa métrica, a estrutura de apoio aproximou um modelo intermediário de um modelo maior. A leitura mais defensável do lema *scaffolding over scale*, à luz destes dados, é portanto condicional: a estrutura de apoio não substitui a escala na capacidade de resolver a tarefa, mas pode substituir parte da escala na disciplina com que o código é entregue, desde que o modelo seja capaz de ler e seguir as regras. O *harness* se mostrou, portanto, irrelevante em um modelo local (gpt-oss) que roda em *hardware* de consumidor, o que mostra que, no mínimo, o modelo deve ser inteligente o suficiente para seguir e armazenar as instruções.

O efeito concentra-se no meio da distribuição. Quem mais ganha são os modelos que partem de uma posição intermediária (DeepSeek e Muse). Os modelos de ponta já escrevem testes e seguem convenções sem instrução, e o *harness* acrescenta pouco. Os modelos mais fracos (gpt-oss-20b e Beam) não respondem: as regras estão na árvore, mas o modelo não as lê

ou não as segue, e a nota de testes permanece em 1,0. Há, assim, um limiar de capacidade abaixo do qual a estrutura de apoio é inerte, o que contraria a versão mais forte da hipótese, segundo a qual o *harness* compensaria justamente os modelos menores.

Qual camada carrega o efeito. A maior parte do ganho de qualidade e do acréscimo de custo aparece já no nível `agents`. O nível `skills` acrescenta um pouco em convenções e testes (os maiores Δ estão nele), o que é coerente com o conteúdo das *skills*, que detalham exatamente essas práticas. O nível `full` não acrescenta nada mensurável e, em alguns casos, devolve parte do ganho; a configuração de LSP, cuja intenção é reduzir leituras, não reduziu o consumo de *tokens* de entrada em relação ao `skills` de forma consistente. Com uma repetição, as diferenças entre os três níveis com *harness* estão dentro do ruído, e a conclusão segura é apenas que a primeira camada responde pela maior parte do efeito.

Custo contra benefício. O *harness* encarece cada execução em dezenas de pontos percentuais em turnos e *tokens* de entrada, para um *pass@1* idêntico. Se o critério for apenas resolver a tarefa medida por testes, o *harness* é custo sem retorno nesta amostra, ou seja, encarece substancialmente a saída sem resolver nenhum problema a mais e, como consequência residual, aumenta consideravelmente o tempo de geração da LLM. Se o critério for entregar um PR com testes e dentro das convenções do projeto, o acréscimo compra um ganho de 0,3 ponto na nota geral e de 0,5 a 0,8 ponto em testes, mais dez casos de teste por mudança, o que, em um fluxo real, reduziria o trabalho de revisão e de complementação posterior. Parte do custo adicional é, aliás, o custo de escrever e executar os testes que o `AGENTS.md` exige. A relação custo-benefício depende do escopo e da relevância de testes extensivos em cada projeto.

Relação com a literatura. O resultado dialoga com o achado de Yang et al. (2024) de que a interface do agente altera a taxa de resolução: aqui, a intervenção foi textual e versionada no repositório, e não na ferramenta, e o seu efeito apareceu na qualidade, não na resolução, em um conjunto de tarefas em que a resolução já estava no teto. A dissociação entre correção funcional e qualidade ecoa Liu et al. (2023) e Pearce et al. (2022), que mostram, por caminhos diferentes, que passar nos testes não esgota o que se espera de um código. Por fim, a concordância entre juízes e a sua correlação com medidas independentes estão em linha com o que Zheng et al. (2023) e Zhuo (2024) relatam, e o cuidado com a autopreferência segue Panickssery, Bowman e Feng (2024).

AMEAÇAS À VALIDADE

O Quadro 4 reúne as ameaças identificadas no desenho do *benchmark*, na rodada reduzida e na análise de qualidade, com a mitigação adotada e o que permanece em aberto. Quatro delas merecem destaque.

Poder estatístico e teto. Com uma repetição por célula e 14 tarefas por modelo, o McNemar exige ao menos seis discordâncias na mesma direção para $p < 0{,}05$: efeitos de 5 a 15

pontos percentuais no *pass@1* passariam despercebidos, e ausência de evidência não é evidência de ausência. Como quatro modelos resolvem 95% ou mais das tarefas sem *harness* e os testes ocultos não discriminam o suficiente entre modelos fortes, só tarefas mais difíceis permitiriam medir ganho de *pass@1* acima do teto; entretanto, tarefas mais complexas ficam fora do escopo possível deste trabalho, assim como do escopo monetário disponível para executar benchmarks em modelos de ponta e tarefas complexas.

Comparações múltiplas e pares correlacionados. Foram feitas 39 comparações de qualidade sem correção. Com um controle da taxa de falsas descobertas (Benjamini e Hochberg, 1995), os efeitos em convenções, testes, casos de teste e nota geral ($p \leq 0{,}001$) resistiriam, mas os de Larastan, complexidade e legibilidade (p entre 0,015 e 0,057) devem ser lidos como sugestivos. As cinco subtarefas dos dois PRs compostos são correlacionadas, o que reduz o número efetivo de pares.

Oráculo e juízes. O oráculo é parcialmente escrito com IA e acopla-se a nomes; os juízes são LLMs sem calibração humana e podem concordar no mesmo erro. O Muse Spark ajudou a escrever o código do repositório-sujeito, treina com os dados enviados e é, ao mesmo tempo, modelo avaliado e juiz, inclusive do próprio código; o controle de autopreferência não encontrou favorecimento líquido, mas uma afinidade de estilo com o repositório não pode ser descartada, e o Muse é o juiz mais generoso justamente com o código do Sonnet e do Haiku.

Generalização. As tarefas vêm de um único repositório privado, de um único autor, em uma única pilha, e o *harness* é o desse repositório, criado manualmente com auxílio de LLM pelo autor unicamente para codificação assistida neste repositório. Nada aqui autoriza generalizar para outros repositórios, linguagens ou *harnesses*, e as comparações entre modelos misturam modelo e *runner*: apenas as comparações de níveis dentro de cada modelo, ou no máximo família de modelos, são controladas.

CONSIDERAÇÕES FINAIS

Este trabalho perguntou se um *harness* em nível de repositório compensa a escala do modelo em agentes de código. Para respondê-la, foi construído um *benchmark* de ablação com 14 tarefas derivadas de *pull requests* reais de uma aplicação Laravel privada, avaliadas por testes ocultos; um orquestrador que executou seis modelos em quatro níveis cumulativos de *harness*, em *sandbox* e com registro reprodutível; e uma análise de qualidade de código em três camadas sobre 309 execuções e 836 revisões de juízes. Os cinco objetivos específicos foram atingidos: o *benchmark*, os níveis e a execução foram realizados como planejado, com a redução de três repetições para uma registrada e justificada; a análise de qualidade e a análise estatística pareada foram conduzidas e seus resultados discutidos.

A resposta à pergunta depende da métrica. Na taxa de resolução, o *harness* não com-

Quadro 4 – Ameaças à validade, mitigação adotada e o que permanece em aberto

Tipo	**Ameaça**	**Mitigação e situação**
Conclusão	Uma repetição por célula; 14 tarefas	As tarefas funcionam como repetição; a variância do mesmo modelo na mesma célula não é estimada (20 células das repetições 2 e 3 da rodada cancelada ficam no manifesto como amostra).
Conclusão	39 comparações sem correção	Número informado; efeitos com $p \leq 0{,}001$ sobrevivem a um controle de falsas descobertas, os demais são sugestivos.
Conclusão	Subtarefas correlacionadas (T03a/b, T04a/b/c)	Registrado; o número efetivo de pares é menor que 70.
Interna	Efeito de teto; testes pouco discriminativos	Análise de qualidade em três camadas como medida complementar; tarefas mais difíceis ficam como trabalho futuro.
Interna	Beam incompleto (29/56, dois *endpoints*); Qwen retirado	Fora das comparações entre níveis; apresentados só de forma descritiva.
Interna	Limite de memória e *watchdog* introduzidos durante a rodada	Células afetadas refeitas como infraestrutura; decisões tomadas sem olhar o resultado.
Interna	gpt-oss quantizado, com o analisador de formato do llama.cpp	24 de 56 falhas são de formato de saída: limitação do conjunto modelo mais servidor, registrada.
Construto	Oráculo parcialmente escrito com IA (cerca de 15% dos *commits*); testes acoplados a nomes	Contrato público no enunciado; acoplamento residual anotado por tarefa.
Construto	Juízes LLM, sem calibração humana; generosidade do Muse concentrada no código de Sonnet e Haiku	Até três juízes por mudança (dois de famílias distintas e o da própria família, com controle de autopreferência); cegamento; evidência obrigatória; κ, viés por juiz e correlação com medidas independentes reportados.
Construto	Muse Spark: coautor do repositório, modelo avaliado e juiz	Julga o próprio código só com dois juízes de fora ao lado, sem autopreferência líquida ($-0{,}40$); checagem de que só o modelo sob teste respondeu; afinidade de estilo não descartada.
Construto	Larastan diverge dos juízes (ρ de $-0{,}53$ no piloto para $-0{,}07$)	Larastan não está instalado no projeto e os agentes não o executam; a aplicação não inicializa em parte das execuções do gpt-oss, o que reduz o n. Em aberto.
Construto	Regras C1 a C10 heurísticas	Validação manual no piloto (25 de 25 violações reais); C5 e C8 só exercitadas na rodada completa, com uma correção em C8.
Externa	Um repositório, um autor, uma pilha, um *harness*	Sem generalização; protocolo público para replicação em outros repositórios.
Externa	*Runners* diferentes entre famílias; internet liberada no *sandbox*; preâmbulo específico	Comparações só dentro de cada modelo; detector de contaminação e repositório privado (0 vazamentos); preâmbulo idêntico para todos.

Fonte: elaborado pelo autor (2026).

pensou a escala: quatro dos seis modelos já resolviam 95% ou mais das tarefas sem nenhuma estrutura de apoio, nenhum nível alterou o *pass@1* de nenhum modelo (McNemar exato, $p = 1{,}0$ em 70 pares por nível) e cada execução ficou mais cara em turnos e *tokens*. Entre modelos, o *pass@1* variou de 11% a 100%; entre níveis de um mesmo modelo, no máximo 7 pontos percentuais. Na qualidade do código, o *harness* teve efeito consistente: a nota geral dos juízes subiu 0,31 a 0,34 ponto, a nota de convenções 0,46 a 0,52 e a de testes 0,60 a 0,87, com cerca de dez casos e 130 a 160 linhas de teste a mais por execução, sem alterar o tamanho do código de produção, menos erros novos de análise estática e menor complexidade, e sem alteração em

requisitos, segurança ou escopo. O efeito concentrou-se nos modelos de capacidade intermediária (DeepSeek v4.1 flash e Muse Spark 1.3), foi pequeno nos modelos de ponta, que já partem alto, e nulo nos modelos mais fracos (gpt-oss-20b e Beam), que não seguem as regras. A primeira camada, as regras sempre ativas, respondeu pela maior parte do efeito e do custo; as *skills* acrescentaram pouco e a configuração de LSP nada mensurável. Em síntese, nesta amostra a escala explica se a tarefa é resolvida e o *harness* explica parte de como o código é entregue.

As contribuições são o *benchmark* e seus enunciados, o orquestrador, o pacote de análise de qualidade com a rubrica e as regras de convenção, e os dados de todas as execuções e revisões, disponíveis publicamente, com exceção dos *commits* do repositório-sujeito, que é privado. Do ponto de vista metodológico, o trabalho mostra que um oráculo de testes, sozinho, não distingue entregas que um revisor distinguiria, e que juízes LLM de três famílias, com evidência obrigatória, controle de autopreferência e validação cruzada por medidas objetivas, podem preencher essa lacuna com concordância substancial.

As limitações são as de um experimento de uma repetição, 14 tarefas de um único repositório privado e efeito de teto: o desenho só detectaria efeitos grandes sobre o *pass@1*, e nada aqui se generaliza para outros repositórios, linguagens ou *harnesses*. O oráculo foi parcialmente escrito com IA, os juízes são LLMs sem calibração humana, as regras de convenção são heurísticas, o Beam ficou incompleto, servindo apenas como prova de que modelos menores ignoram as instruções do *harness*, e as comparações múltiplas não foram corrigidas.

Como trabalhos futuros, propõem-se: executar mais repetições por célula, para estimar a variância de um mesmo modelo e ganhar poder estatístico; selecionar tarefas mais difíceis, fora do teto, nas quais um ganho de *pass@1* seja mensurável, assim como ganho de qualidade entre modelos fortes em tarefas mais complexas; repetir o protocolo em outros repositórios e pilhas, com seus próprios *harnesses*, para testar a generalidade dos achados; fazer a ablação de partes do `AGENTS.md` (apenas a regra de testes, apenas as convenções, apenas a política de segurança), para identificar quais frases carregam o efeito; e calibrar os juízes LLM com revisores humanos em uma amostra das mudanças, para medir a validade das notas diretamente. Uma rodada própria para os modelos locais e a conclusão da matriz do Beam completariam o quadro dos modelos de menor capacidade, onde a hipótese *scaffolding over scale* tem mais a provar.

DECLARAÇÃO SOBRE O USO DE IA GENERATIVA

Ferramentas de inteligência artificial generativa foram utilizadas neste trabalho de quatro formas distintas, descritas também na subseção 2.7.

Primeiro, modelos de linguagem foram o **objeto** do experimento: Claude Sonnet 5.5 e Claude Haiku 5.5 (Anthropic, via Claude Code 2.1.312), Muse Spark 1.3, DeepSeek v4.1 flash e Beam (via OpenCode v2.2.0) e gpt-oss-20b (OpenAI, executado localmente com llama.cpp)

escreveram o código avaliado.

Segundo, a ferramenta Claude Code, desenvolvida pela Anthropic, com o modelo Claude Opus 5.5, foi utilizada na construção do orquestrador, dos *scripts* de análise e dos relatórios intermediários do *benchmark*, sob direção e revisão do autor. O Muse Spark foi utilizado como ferramenta na criação de parte do código do repositório-sujeito, anterior a este trabalho.

Terceiro, os modelos Claude Opus 5.5 (Anthropic), Muse Spark 1.3 e DeepSeek v4.1 flash atuaram como **juízes** na terceira camada da análise de qualidade de código, inclusive sobre o código da própria família, como controle de autopreferência, sob as salvaguardas descritas na subseção 4.5.

Quarto, o texto deste artigo foi redigido com o auxílio do modelo Claude Fable 5.1, desenvolvido pela Anthropic, a partir dos relatórios, dos dados e da documentação produzidos durante a pesquisa, e foi integralmente revisado pelo autor. Todo o conteúdo produzido com o auxílio dessas ferramentas foi analisado, revisado e validado pelo autor, que assume integral responsabilidade pelo conteúdo desta publicação.

REFERÊNCIAS BIBLIOGRÁFICAS

AGENTS.md. *AGENTS.md: a simple, open format for guiding coding agents*. 2025. <https://agents.md/>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 12 e 20.

AKITA, F. *Hot take: harness, loop engineering, graph engineering are bullshit*. 2026. <https://akitaonrails.com/en/2026/08/18/hot-take-harness-loop-engineering-graph-engineering-are-bullshit/>. Acesso em: 2 nov. 2026. Citado 2 vezes nas páginas 12 e 20.

Anthropic. *Claude Code: best practices for agentic coding*. 2025. <https://www.anthropic.com/engineering/claude-code-best-practices>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 12 e 20.

Anthropic. *Effective context engineering for AI agents*. 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>. Acesso em: 29 out. 2026. Citado na página 20.

Anthropic. *Claude Code documentation*. 2026. <https://docs.anthropic.com/en/docs/claude-code>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 12 e 20.

Anthropic. *Prompting Claude Opus 5.5*. 2026. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/prompting-claude-opus-5-5>. Acesso em: 2 nov. 2026. Citado na página 20.

AUSTIN, J. et al. *Program synthesis with large language models*. 2021. ArXiv:2108.07732. Disponível em: <https://arxiv.org/abs/2108.07732>. Acesso em: 29 out. 2026. Citado na página 19.

BECK, K. *Test-Driven Development: by example*. Boston: Addison-Wesley, 2002. Citado na página 22.

BENJAMINI, Y.; HOCHBERG, Y. *Controlling the false discovery rate: a practical and powerful approach to multiple testing*. 1995. Journal of the Royal Statistical Society: Series B (Methodological), v. 57, n. 1, p. 289–300. Citado na página 40.

CHEN, M. et al. *Evaluating large language models trained on code*. 2021. ArXiv:2107.03374. Disponível em: <https://arxiv.org/abs/2107.03374>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 16 e 19.

COHEN, J. Weighted kappa: nominal scale agreement with provision for scaled disagreement or partial credit. *Psychological Bulletin*, v. 70, n. 4, p. 213–220, 1968. Citado na página 17.

DUBOIS, Y. et al. *Length-controlled AlpacaEval: a simple way to debias automatic evaluators*. 2024. ArXiv:2404.04475. Disponível em: <https://arxiv.org/abs/2404.04475>. Acesso em: 29 out. 2026. Citado na página 21.

GERGANOV, G. et al. *llama.cpp: LLM inference in C/C++*. 2026. <https://github.com/ggml-org/llama.cpp>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 16 e 21.

GIL, A. C. *Como elaborar projetos de pesquisa*. 6. ed. São Paulo: Atlas, 2017. Citado na página 14.

HOFFMANN, J. et al. *Training compute-optimal large language models*. 2022. ArXiv:2203.15556. Disponível em: <https://arxiv.org/abs/2203.15556>. Acesso em: 29 out. 2026. Citado na página 21.

ISO/IEC. *ISO/IEC 25010:2011. Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): System and software quality models*. 2011. Geneva: International Organization for Standardization. Citado na página 22.

JIMENEZ, C. E. et al. SWE-bench: can language models resolve real-world GitHub issues? In: *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. Vienna: [s.n.], 2024. Citado 2 vezes nas páginas 12 e 19.

KAPLAN, J. et al. *Scaling laws for neural language models*. 2020. ArXiv:2001.08361. Disponível em: <https://arxiv.org/abs/2001.08361>. Acesso em: 29 out. 2026. Citado na página 21.

LANDIS, J. R.; KOCH, G. G. The measurement of observer agreement for categorical data. *Biometrics*, v. 33, n. 1, p. 159–174, 1977. Citado 2 vezes nas páginas 17 e 35.

Larastan. *Larastan: PHPStan extension for Laravel*. 2026. <https://github.com/larastan/larastan>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 16 e 22.

LIU, J. et al. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. New Orleans: [s.n.], 2023. Citado 2 vezes nas páginas 19 e 39.

LIU, N. F. et al. Lost in the middle: how language models use long contexts. *Transactions of the Association for Computational Linguistics*, v. 12, p. 157–173, 2024. Citado na página 20.

MARTIN, R. C. *Clean Code: a handbook of agile software craftsmanship*. Upper Saddle River: Prentice Hall, 2008. Citado na página 22.

MCCABE, T. J. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2, n. 4, p. 308–320, 1976. Citado na página 22.

MCNEMAR, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, v. 12, n. 2, p. 153–157, 1947. Citado na página 17.

OpenAI. *Introducing SWE-bench Verified*. 2024. <https://openai.com/index/introducing-swe-bench-verified/>. Acesso em: 29 out. 2026. Citado na página 19.

OpenAI. *gpt-oss-120b & gpt-oss-20b model card*. 2025. ArXiv:2508.10925. Disponível em: <https://arxiv.org/abs/2508.10925>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 16 e 21.

OpenCode. *OpenCode documentation*. 2026. <https://opencode.ai/docs>. Acesso em: 29 out. 2026. Citado 2 vezes nas páginas 12 e 20.

PANICKSSERY, A.; BOWMAN, S. R.; FENG, S. LLM evaluators recognize and favor their own generations. In: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Vancouver: [s.n.], 2024. Citado 2 vezes nas páginas 21 e 39.

PEARCE, H. et al. Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions. In: *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*. San Francisco: [s.n.], 2022. p. 754–768. Citado 2 vezes nas páginas 22 e 39.

SHRIVASTAVA, D.; LAROCHELLE, H.; TARLOW, D. Repository-level prompt generation for large language models of code. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Honolulu: [s.n.], 2023. Citado na página 19.

SPEARMAN, C. The proof and measurement of association between two things. *The American Journal of Psychology*, v. 15, n. 1, p. 72–101, 1904. Citado na página 17.

WANG, P. et al. Large language models are not fair evaluators. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Bangkok: [s.n.], 2024. Citado na página 21.

WANG, X. et al. *OpenHands: an open platform for AI software developers as generalist agents*. 2024. ArXiv:2407.16741. Disponível em: <https://arxiv.org/abs/2407.16741>. Acesso em: 29 out. 2026. Citado na página 19.

WILCOXON, F. Individual comparisons by ranking methods. *Biometrics Bulletin*, v. 1, n. 6, p. 80–83, 1945. Citado na página 17.

WILSON, E. B. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, v. 22, n. 158, p. 209–212, 1927. Citado na página 17.

WOHLIN, C. et al. *Experimentation in software engineering*. Berlin: Springer, 2012. Citado na página 14.

YANG, J. et al. SWE-agent: agent-computer interfaces enable automated software engineering. In: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. Vancouver: [s.n.], 2024. Citado 2 vezes nas páginas 19 e 39.

YAO, S. et al. ReAct: synergizing reasoning and acting in language models. In: *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. Kigali: [s.n.], 2023. Citado na página 20.

ZHANG, F. et al. RepoCoder: repository-level code completion through iterative retrieval and generation. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Singapore: [s.n.], 2023. Citado na página 19.

ZHENG, L. et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. New Orleans: [s.n.], 2023. Citado 2 vezes nas páginas 21 e 39.

ZHUO, T. Y. ICE-Score: instructing large language models to evaluate code. In: *Findings of the Association for Computational Linguistics: EACL 2024*. St. Julian's: [s.n.], 2024. Citado 2 vezes nas páginas 21 e 39.

APÊNDICE A – AS 14 TAREFAS DO *BENCHMARK*

O Quadro 5 lista as tarefas da rodada reduzida. O porte segue o número de arquivos de produção da solução de referência (pequena até 10, média de 11 a 25, grande acima de 25). A coluna Arq./lin. traz o tamanho da solução de referência (arquivos alterados e linhas inseridas, incluindo testes). As tarefas T03a e T03b são subtarefas do PR de autenticação e T04a, T04b e T04c são subtarefas do PR de entidades; os PRs compostos não entram no conjunto. Duas tarefas (T10 e T11) vêm de *branches* não mescladas.

Quadro 5 – Tarefas do *benchmark*, com categoria, porte, origem, tamanho da referência e observações

Id	Título	Categ.	Porte	Origem	Arq./lin.	Observações
T01	Notificador de alertas no Discord e canais de *log*	ops	pequena	PR 17	10/400	Bem especificada; 2 testes ocultos novos.
T02	Menção a um usuário configurado em todo alerta	ops	pequena	PR 30	6/103	Muito pequena; modifica um teste existente; item de calibração.
T03a	Autenticação de sessão (login, registro, redefinição, verificação)	auth	grande	PR 19 (sub.)	31/1 062	Subtarefa de T03; 3 testes novos; acoplamento a nomes de rota.
T03b	Redefinição de senha e e-mail transacional com tema	auth	pequena	PR 19 (sub.)	11/584	Subtarefa de T03; 1 teste novo; chaves de tradução.
T04a	Entidades, associações com papéis e *policies*	tenancy	média	PR 31 (sub.)	15/768	Subtarefa de T04; 4 testes novos; coluna gerada no banco.
T04b	Entidades mais *onboarding*, sessão de locatário e seletor	tenancy	média	PR 31 (sub.)	33/1 566	Inclui os testes de T04a; acoplamento de marcação.
T04c	Entidades mais painel, gestão de membros e configurações	tenancy	média	PR 31 (sub.)	28/1 831	Inclui os testes de T04a; testes de adulteração em Livewire.
T05	Associação por convite com e-mail e modal de membro	tenancy	média	PR 34	24/1 456	Modifica um teste existente; exige remover comportamento antigo.
T06	Filas, senhas e ação de emissão de senha	domain	média	PR 33	14/679	14 testes de domínio, sem UI; sequência de códigos e escrita única.
T07	Chamada da próxima senha com modos e justiça de 3 chamadas	domain	pequena	PR 37	6/390	15 testes; algorítmica; desempate por id.
T08	Ações de gestão de filas, permissões por fila e *policy*	domain	média	PR 39	17/1 149	Modifica um teste existente ao exigir autorização.
T09	Telas Livewire de gestão de filas e painel do atendente	UI	média	PR 40	15/2 013	Grande superfície de UI; acoplamento de marcação; 2 testes modificados.
T10	API BFF do totem (pareamento, JWT, filas, prioridades, emissão)	API	grande	*branch* 42[a]	38/1 468	Dependência JWT; CORS; códigos de erro exatos.
T11	Prefixos de código por prioridade com sequências diárias	domain	pequena	*branch* 46[a]	14/518	Modifica 2 testes existentes; muitos casos de borda.

Fonte: elaborado pelo autor (2026). [a] *Branch* não mesclada (tarefa *held-out*).

APÊNDICE B – RUBRICA DOS JUÍZES

Cada juiz recebe o enunciado, as seções normativas do `AGENTS.md`, o *diff* da mudança, a rubrica e um esquema JSON de saída. Ele atribui a cada dimensão uma nota de 1 a 5, com evidência obrigatória (`caminho:linha` ou trecho citado), e uma nota geral que responde à pergunta "você aprovaria este *pull request*?" (5: aprova como está; 4: aprova com observações; 3: precisa de mudanças; 2: precisa de retrabalho substancial; 1: rejeita). O juiz também lista os problemas encontrados, com severidade, dimensão, local e explicação, e declara a confiança na própria revisão. O Quadro 6 resume as âncoras; as notas 2 e 4 são usadas para casos intermediários. O juiz é instruído a não recompensar o comprimento, a não comparar com outras mudanças e a não tentar adivinhar quem escreveu o código.

Quadro 6 – Dimensões da rubrica e âncoras das notas 5, 3 e 1

Dimensão	**5**	**3**	**1**
Requisitos	Todos os requisitos e o contrato atendidos; casos de borda tratados	Caminho principal feito; algum requisito, caso de borda ou detalhe do contrato ausente ou errado	Maioria dos requisitos ausente, contrato errado ou código que não funciona
Convenções	Segue toda convenção aplicável (ações para escritas de domínio, `#[Validate]`, *policies*, migrações geradas, `$fillable`, componentes Blade, Bulma, docs em sincronia)	Segue a maioria; um ou dois desvios claros	Ignora as convenções (camadas de serviço, lógica em *views*, padrões refeitos à mão)
Segurança	Nenhuma fraqueza; autorização e validação onde necessárias	Uma lacuna de impacto limitado	Falha explorável (escrita não autorizada, saída sem escape, confiança em ids do cliente)
Testes	Testes cobrem todos os requisitos e os caminhos de falha e autorização	Alguns testes, só caminho feliz, ou casos importantes ausentes	Nenhum teste para o novo comportamento, ou testes enfraquecidos ou apagados
Simplicidade	Enxuto e direto; reutiliza o que existe	Alguma duplicação ou indireção desnecessária	Sobre-engenharia pesada ou cópia e cola
Legibilidade	Fácil de ler e de mudar	Compreensível com esforço; métodos longos ou nomes pouco claros	Difícil de seguir; código morto, saída de depuração, estilo inconsistente
Escopo	Tudo no *diff* serve à tarefa	Algumas edições não relacionadas	Grandes mudanças ou funcionalidades não pedidas

Fonte: elaborado pelo autor (2026).

APÊNDICE C – REGRAS DE CONVENÇÃO C1 A C10

As dez regras da camada 2 foram derivadas do `AGENTS.md` e verificadas automaticamente no *patch* de cada execução. Cada regra só se aplica quando a mudança toca o seu assunto (por exemplo, C2 e C3 só se aplicam quando há um componente Livewire no *diff*), e o escore da execução é a fração das regras aplicáveis cumpridas. Toda violação guarda a evidência que a motivou. A Tabela 11 lista as regras e, para as 309 execuções da rodada, quantas vezes cada uma se aplicou e foi cumprida.

Tabela 11 – Regras de convenção, número de execuções em que cada uma se aplicou e em que foi cumprida (309 execuções)

Regra	**Descrição**	**Aplica**	**Cumpre**
C1	Código de produção acompanhado de teste (*test-first*)	308	193
C2	Componentes Livewire validam com `#[Validate]` e `$this->validate()`	106	53
C3	Toda ação pública de Livewire é autorizada no servidor	87	36
C4	Escritas de domínio passam por classes em `app/Actions`	148	39
C5	Migrações geradas (nome com *timestamp*) e reversíveis (`down()`)	171	168
C6	Nenhuma camada de serviço ou repositório	261	260
C7	Blade escapa a saída (`{{ }}`, nunca `{!! !!}`)	130	130
C8	Modelos novos declaram `$fillable` ou `$guarded`	151	84
C9	Interface em Bulma, sem classes utilitárias de CSS nem bibliotecas de UI em JavaScript	130	129
C10	Autorização por *policies* ou *gates*, não por verificação manual de papel	149	106

Fonte: elaborado pelo autor (2026).

APÊNDICE D – DADOS COMPLEMENTARES DA RODADA REDUZIDA

Este apêndice reúne as medianas absolutas de custo por execução, que a Figura 5 apresenta de forma relativa, e a linha do tempo dos incidentes e decisões da rodada reduzida, mencionada na subseção 4.3.

Tabela 12 – Custo por execução (medianas) por modelo e nível

Modelo	Nível	Turnos	Tokens de saída	Tokens de entrada	Tempo (min)
Sonnet 5.5	`none`	18	22 502	905 810	3,1
	`agents`	30	30 112	1 468 970	4,1
	`skills`	29	31 398	1 507 708	4,5
	`full`	26	27 909	1 363 034	4,6
Haiku 5.5	`none`	62	73 469	3 767 150	5,9
	`agents`	70	86 608	5 397 860	8,3
	`skills`	74	85 562	6 428 217	9,0
	`full`	71	87 200	5 525 890	8,1
Muse Spark 1.3	`none`	62	20 360	4 125 950	4,1
	`agents`	64	18 510	4 354 079	8,6
	`skills`	74	18 452	4 700 618	5,1
	`full`	67	19 244	4 989 247	5,8
DeepSeek v4.1 flash	`none`	58	17 518	4 256 746	9,3
	`agents`	68	20 720	4 962 629	8,7
	`skills`	67	20 570	6 097 903	8,3
	`full`	76	22 686	6 913 314	8,4
gpt-oss-20b	`none`	42	10 990	693 686	2,5
	`agents`	38	10 620	691 718	1,8
	`skills`	46	13 660	977 582	2,4
	`full`	46	11 222	982 754	2,3

Fonte: elaborado pelo autor (2026). Tokens de entrada incluem leitura de *cache*. O tempo é o de parede do agente e não é comparável entre modelos.

Quadro 7 – Incidentes da rodada reduzida (29 de outubro de 2026) e decisões tomadas

Hora	Evento	Decisão
00h37	Detector de contaminação marca um *link* para um PR escrito no `CHANGELOG.md`	*Links* para o repositório-sujeito deixam de contar; o veredito passa a ser recalculado do *log* bruto completo; 2 falsos positivos voltam aos dados.
00h44 e 02h54	DeepSeek tenta ler o repositório-sujeito no GitHub (arquivo bruto; depois a API)	Repositório privado, sem credencial no *sandbox*: 404, sem vazamento. Tentativas contadas à parte como comportamento observado.
01h24	Contêiner de um agente chega a 23 GB de *swap* e é morto pelo sistema	Limite de 6 GB de memória por contêiner; a célula é refeita como infraestrutura.
01h52	Qwen 3.6 35B-A3B leva cerca de 23 minutos por célula na GPU única	Modelo retirado da rodada (restariam cerca de 20 horas); 3 células ficam só como nota.
02h35	Provedor do Beam fora do ar: agentes mudos por 10 a 15 minutos até o *timeout*	*Watchdog* de silêncio (8 minutos) trata o travamento como limite de taxa; 6 falsos *timeouts* reenfileirados, nunca contados como dado.
03h22	CLI do OpenCode sai com erro depois de o agente terminar	Classificado como infraestrutura e refeito; a decisão não dependeu do resultado.
07h08 a 07h45	Beam retomado por outro *endpoint* do mesmo modelo; cota gratuita esgota após 6 células	Beam encerrado com 29 das 56 células; apresentado só de forma descritiva.

Fonte: elaborado pelo autor (2026).